

UNIVERSIDAD DE SANTIAGO DE
COMPOSTELA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA

**Optimización en el acceso a nubes de
puntos de 3D mediante la utilización de
octrees y coordenadas polares**

Autor/a:

Manuel Ramallo Blanco

Tutores:

Francisco M. Fernández Rivera

Miguel Yermo García

Grado en Ingeniería Informática

Junio 2026

Trabajo de Fin de Grado presentado en la Escuela Técnica Superior de
Ingeniería de la Universidad de Santiago de Compostela para la obtención del
Grado en Ingeniería Informática

Agradecimentos

Se se quiere pór algún agradecemento, este vai aquí.

Resumen

En este trabajo se describe y se presenta una solución de mejora de eficiencia temporal en el acceso a nubes de puntos 3D utilizando como estructura de datos *octrees* lineales. El estudio consiste en el reordenamiento espacial de puntos para tratar de explotar la capacidad de poda a la hora de encontrar vecindades. Se evalúan resultados temporales usando diferentes técnicas de reordenamiento, operando tanto con las coordenadas cartesianas como con las coordenadas polares de los puntos. Los resultados experimentales muestran un impacto positivo en la eficiencia temporal bajo determinadas circunstancias, a cambio de un gasto en memoria utilizada.

Indice

Índice de figuras

Índice de cuadros

Capítulo 1

Introducción

Las nubes de puntos 3D son una representación de una realidad, como un objeto o un terreno, mediante un conjunto de puntos en el espacio tridimensional. Cada uno de esos puntos está caracterizado por sus coordenadas espaciales, así como otras propiedades como el color o la intensidad. Entre las aplicaciones más importantes de la elaboración de estas nubes están la cartografía, la elaboración de modelos tridimensionales en CAD de piezas fabricadas, la metrología, la arqueología o la conducción autónoma. La tecnología predominante para la captura de estos puntos es la LiDAR (Light Detection And Ranging).

Una de las tareas más comunes al trabajar sobre nubes de puntos 3D es la búsqueda de vecindades alrededor de un punto dado. Una vecindad se define como el conjunto de puntos que están dentro de una figura geométrica determinada en el espacio con radio R y con el centro en el punto para el cual se están buscando vecinos. Esta combinación de figura geométrica y de radio se conoce como kernel de búsqueda. Otro método alternativo de análisis de vecindades es buscar los K puntos más cercanos al elemento de partida, técnica llamada KNN (*k nearest neighbors*). Cabe destacar, eso sí, que el presente trabajo se centra exclusivamente en el primer enfoque (búsquedas de geometría de kernel), quedando las búsquedas KNN fuera de alcance.

Un gran problema a nivel de hardware de este proceso sobre nubes de puntos es que estos datos se recogen sin ningún tipo de estructura ni orden basado en localidad. Este fenómeno se junta con el hecho de que estas nubes pueden tener millones de puntos almacenados. Por eso, se necesitan métodos eficientes para acelerar este proceso, intrínsecamente costoso, de acceso a puntos cercanos.

Los *Octrees* y los *KDTrees* son estructuras jerárquicas ampliamente utilizadas para lidiar con este problema de ineficiencia. Consisten en dividir la nube de puntos de manera recursiva en zonas próximas en el espacio físico. Los *Octrees* son estructuras donde cada nodo se divide exactamente en 8 hijos, dividiendo cada dimensión a la mitad (solo se usa en espacios tridimensionales). Los puntos pertenecen al nodo hoja que los contiene en su interior. Por otra parte, los *KDTrees* son árboles binarios que dividen cada nodo interno solamente según una dimensión. En cada nivel se elige un eje y se traza un hiperplano que divide a los puntos en dos nodos. De esta forma, este árbol es generalizable para cualquier espacio de dimensiones.

El rendimiento de estas estructuras depende directamente de la distribución de los puntos. Por eso es muy útil la aplicación de técnicas de reordenamiento espacial basadas en curvas de llenado de espacio (SFC). Su objetivo es mejorar la coherencia entre la localidad espacial de los puntos y su disposición en memoria para optimizar los accesos en el cálculo de puntos vecinos.

En este trabajo se usará como base el modelo de *Octrees* propuesta por Viñambres et al.[?], donde se aplican las reordenaciones espaciales de Morton y Hilbert sobre diferentes *Octrees* implementados en el lenguaje C++. Se construyó tanto una estructura clásica basada en punteros enlazados como una versión basada en almacenamiento en un vector lineal. Sobre estas estructuras, y los algoritmos de búsqueda de vecindades asociados a ellas, se tratará de optimizar todavía más el acceso a puntos cercanos. Para ello se empleará la técnica algorítmica de la poda.

Esta estrategia, de forma general, consiste en descartar o *podar* regiones del espacio (nodos) o conjuntos de datos para los que matemáticamente se puede demostrar que no satisfacen las condiciones necesarias para ser considerados solución. Su aplicación en este contexto es ser capaz de conocer *a priori* qué partes del *Octree* no pueden contener ningún vecino para una búsqueda dada. Esta técnica es, de hecho, el motivo por el que los *Octrees* (y los KDTrees) son estructuras ideales para almacenar nubes de puntos 3D. Es muy sencillo podar nodos comprobando si sus límites están fuera del alcance del kernel de vecindad. Sin embargo, en este trabajo se explorará la poda a nivel de puntos dentro de un nodo, es decir, se busca ser capaz de descartar puntos pertenecientes a un nodo hoja sin tener que evaluarlos individualmente. De esta forma, se acotaría un rango específico dentro del dominio espacial del nodo y se rechazarían sin necesidad de recorrerlos secuencialmente todos los puntos de la hoja que no perteneciesen a ese rango. Para conseguir esta tarea se realizarán reordenamientos de puntos a nivel de hoja siguiendo diversos criterios basados en coordenadas cartesianas con ejes X, Y, Z como definidores de los puntos, pero también se manejarán sistemas alternativos de representaciones tridimensionales: sistemas esféricos y cilíndricos. Estos sistemas son expansiones a espacios tridimensionales de las coordenadas polares.

Las coordenadas polares son un sistema de coordenadas bidimensional donde cada punto es caracterizado por una distancia radial y un ángulo con respecto del origen. Pueden ser expandidas a un espacio tridimensional mediante dos sistemas ligeramente distintos: las coordenadas esféricas y las cilíndricas.

Los reordenamientos en cuestión consisten en almacenar los puntos dentro de cada nodo terminal ordenados según alguna característica de las mencionadas. Así, se podrán calcular los límites de forma teórica para ese valor que puede tener un punto de forma que satisfaga la restricción de vecindad, dada una búsqueda con un kernel determinado.

Para resumir, esta propuesta de algoritmo y este estudio se pueden descomponer en una serie de objetivos claros, que se definen a continuación:

1. Usar las técnicas de los reordenamientos y las podas para mejorar la eficiencia en las búsquedas de vecinos mediante kernels geométricos sobre nubes de puntos 3D capturadas por sensores LiDAR. Al hablar de

eficiencia se hace referencia a la temporal, buscando la mayor rapidez posible. En este trabajo no se tomará como prioridad el ahorro en memoria computacional, aunque como es lógico sí se tendrá en cuenta y se tratará de minimizar su gasto siempre que no genere un impacto negativo en el rendimiento temporal.

2. Comprobar la validez teórica de la estrategia de los reordenamientos y podas a nivel de hoja en *Octrees*, de forma que se pueda considerar una técnica plausible, más allá de los resultados temporales obtenidos en esta implementación del problema.
3. Estudiar y analizar el rendimiento de los algoritmos de búsqueda optimizados mediante la citada estrategia frente a los algoritmos de búsqueda de vecindades de partida, de forma que se puedan sacar conclusiones lo más realistas posibles de la efectividad de este proyecto.
4. Analizar la escalabilidad de la paralelización en los nuevos algoritmos de búsqueda, comparando este factor también con los algoritmos de partida.

Capítulo 2

Estado del arte

El uso de Octrees como estructura de datos en la gestión y almacenamiento de nubes de puntos 3D es en la actualidad predominante a nivel global. Existen librerías de software libre ampliamente reconocidas que proporcionan soporte nativo tanto a estructuras Octrees como a KD-Trees para optimizar la indexación espacial.

Una de estas librerías es Open3D [?]. Está escrita en C++ pero cuenta con interfaces estables para numerosos lenguajes, considerándose de facto el estándar en Python para la manipulación y visualización 3D. Open3D incluye métodos avanzados para el cálculo de distancias y la búsqueda de vecindad, dando soporte tanto a consultas basadas en kernels de radio como a la técnica de los K vecinos más cercanos (*K-Nearest Neighbors* o KNN).

Otro gigante en el manejo de nubes de puntos, considerada la librería de referencia absoluta en el entorno académico e industrial para el procesamiento de geometría 3D en C++, es PCL (*Point Cloud Library*) [?]. PCL proporciona un ecosistema masivo y modular que da soporte nativo a estructuras espaciales mediante las clases `pcl::Octree` y `pcl::KdTree`. El enfoque de PCL en sus Octrees tradicionales se basa en una estructura dinámica de punteros. Este diseño otorga una gran versatilidad para realizar modificaciones en tiempo real sobre la nube, como la inserción o eliminación dinámica de puntos y la detección de cambios espaciales. Por el contrario, introduce una severa penalización en el rendimiento de la memoria. Esto se debe a la incoherencia entre la localidad espacial de los datos y su disposición física en el *heap* o montón, provocando constantes fallos de caché y saltos de punteros en el direccionamiento de la CPU durante los recorridos de búsqueda.

Un aspecto crítico que comparten tanto Open3D como PCL es que no implementan sus algoritmos de búsqueda desde cero, sino que integran internamente la librería FLANN (*Fast Library for Approximate Nearest Neighbors*) [?]. FLANN es una biblioteca altamente optimizada para la búsqueda de vecinos más cercanos en espacios vectoriales multidimensionales. Si bien su principal característica es la capacidad de realizar búsquedas aproximadas (*Approximate Nearest Neighbor* o ANN) para mitigar la maldición de la dimensionalidad sacrificando un margen mínimo de precisión, en el contexto de búsquedas exactas por radio o kernel (como las evaluadas en este estudio) se configura en su modo

exacto para operar como línea base de comparación directa.

Otras librerías relevantes con el mismo propósito, escritas también en C++, son PicoTree [?] y NanoFLANN [?], ambas basadas en *KD-Trees*. PicoTree genera la estructura integrada en un array contiguo mediante la indexación previa de los puntos, permitiendo búsquedas esféricas y de KNN altamente eficientes. Por su parte, NanoFLANN, que es un *fork* de la librería FLANN hecha para optimizar el gasto en memoria y la eficiencia en búsquedas por radio, además de ser flexible para operaciones en 2 y 3 dimensiones.

Por otro lado, las curvas de llenado del espacio (SFC) [?] son ampliamente aplicadas a estructuras jerárquicas para tareas de indexado o compresión. El funcionamiento de las SFC consiste en representar un hipercubo N -dimensional mediante una línea continua. En el caso de las nubes de puntos 3D nos interesa mapear los puntos tridimensionales a una representación unidimensional. Para lograr este reordenamiento, en la literatura destacan dos estrategias diferentes, las curvas de Morton y las curvas de Hilbert.

La curva de Morton basa su estrategia en el entrelazado de bits de las coordenadas binarias de los puntos, dividiendo el espacio en cuadrantes de forma recursiva. Al alternar los bits asociados a cada dimensión, se consigue un índice único para cada región en el espacio tridimensional. La ventaja de esta aproximación radica en su eficiencia computacional y simplicidad algorítmica, ya que la codificación y decodificación se realizan mediante operaciones a nivel de bits en la CPU. Sin embargo, introduce discontinuidades en los límites de los cuadrantes y esto provoca incoherencias en cuanto a localidad en memoria de puntos próximos.

Por el contrario, la curva de Hilbert se define como una curva fractal continua. Esta resuelve el problema de las discontinuidades presente en los códigos de Morton, garantizando que aquellos puntos que se encuentran adyacentes en el espacio tridimensional mantengan una proximidad estricta en la representación lineal. Para lograr esta continuidad física, el algoritmo hace uso de rotaciones de los cuadrantes espaciales de forma recursiva. Aunque esta propiedad ofrece una preservación de la localidad espacial muy superior, su cálculo matemático conlleva una mayor complejidad algorítmica y un coste computacional superior al de la curva de Morton.

Recientemente, la integración de códigos de Morton en estructuras de Octree lineales ha sido una estrategia clave para mejorar la eficiencia en la ordenación de puntos. Un ejemplo de ello es el trabajo de Behley et al. con la librería *unibn* [?], donde se emplean estos códigos para reducir la necesidad de recorrer el árbol en toda su profundidad durante las búsquedas de vecindad, logrando superar el rendimiento de otras herramientas populares de la disciplina.

No obstante, la aplicación directa de SFC en la búsqueda de vecindades en nubes de puntos 3D obtenidas con sensores LiDAR fue implementada originalmente en el trabajo de Viñambres et al. [?]. En dicha investigación se analiza en primer lugar una estructura Octree de punteros estándar construida mediante fragmentaciones recursivas. A pesar de que a esta estructura se le pueden aplicar reordenamientos según curvas de Morton o Hilbert para mejorar la cercanía en memoria de los puntos próximos, el enfoque clásico sigue adoleciendo del problema de los accesos indexados: cada transición entre nodos durante el

recorrido implica saltar a una dirección del mapa de memoria que puede estar totalmente dispersa.

Por este motivo, los autores propusieron e implementaron una versión lineal del Octree, la cual almacena todos los puntos de la nube en un único vector contiguo, garantizando que todos los elementos pertenecientes a un mismo nodo hoja residan en posiciones físicas consecutivas del array. Adicionalmente, se introdujo un algoritmo de búsqueda por kernel geométrico que optimiza el proceso de recorrido: a diferencia del método clásico que desciende recursivamente verificando la mera intersección, este algoritmo identifica de forma temprana si el volumen del kernel contiene por completo el área que cubre el nodo (sea este interno o terminal). De cumplirse esta condición de inclusión total, el algoritmo realiza un volcado secuencial directo de todos los puntos del subárbol, abortando el descenso.

En la publicación de Viñambres et al. [?] se presentan resultados experimentales que demuestran la superioridad del Octree lineal (en sus variantes con reordenamiento de Morton y Hilbert) frente al Octree clásico de punteros. Asimismo, se incluye un exhaustivo análisis comparativo de rendimiento frente a las soluciones de código abierto de referencia mencionadas anteriormente, tales como los módulos de PCL, PicoTree, NanoFLANN y la librería *unibn*. Bajo diferentes tipologías de datasets y variaciones en el radio del kernel de búsqueda, el Octree lineal reportó, de manera sistemática, los menores tiempos de ejecución computacional.

Sobre este escenario técnico se fundamenta el presente trabajo. En la práctica totalidad de las librerías del estado del arte existe un patrón de diseño común: las estructuras justifican su eficiencia basándose exclusivamente en la aplicación de la poda a nivel de nodos intermedios (ramas). Mediante funciones de solapamiento geométrico, se descartan de forma temprana subárboles completos cuyos volúmenes no intersecan con el kernel de consulta. Sin embargo, una vez que el algoritmo alcanza un nodo hoja válido, se ve obligado a evaluar la distancia de todos los puntos contenidos en él de manera individual y secuencial.

Hasta donde se tiene constancia en la literatura técnica y en las soluciones de código abierto analizadas, no existe ninguna implementación que acote el rango de puntos a evaluar mediante una poda matemática interna que actúe de forma nativa en el interior del propio nodo hoja. Este cuello de botella constituye la principal motivación y el núcleo de la contribución de este trabajo.

Capítulo 3

Materiales

En este capítulo se definirán y precisarán todas las herramientas, métodos, algoritmos y equipos que fueron empleados y tomados para la realización del estudio.

3.1. Equipos de cómputo y entorno de desarrollo

El entorno hardware en el que fueron desarrollados los experimentos de este proyecto es el Centro de Supercomputación de Galicia (CESGA). Este centro es un nodo de la Red Española de Supercomputación y datos (RES) y cuenta con una capacidad de computación total de 4.36 PetaFLOPS, estando compuesto por 357 nodos interconectados a través de una red Infiniband HDR.

Existen diferentes grupos de nodos con distintas especificaciones hardware. En concreto, estos experimentos fueron realizados en un nodo del tipo ilk, grupo formado por un total de 256 nodos de cómputo con las especificaciones detalladas a continuación.

Cuadro 3.1: Especificaciones técnicas del entorno de ejecuciones [?].

Componente	Detalle y Configuración
Procesador (CPU)	2× Intel Xeon <i>Ice Lake</i> 8352Y
Núcleos de Cómputo	32 núcleos por procesador (64 cores/nodo)
Memoria RAM	256 GB (247 GB disponibles para uso real)
Almacenamiento Local	960 GB SSD NVMe
Interconexión	1× InfiniBand HDR 100

El entorno de software configurado y desplegado sobre esta arquitectura de hardware consta de las siguientes especificaciones:

- **Sistema Operativo:** Distribución Rocky Linux versión 8.4 (*Green Obsidian*), compatible con entornos Red Hat Enterprise Linux (RHEL).
- **Estándar de Lenguaje:** Se empleó el estándar **C++20**.
- **Compilador:** Las implementaciones fueron compiladas mediante la *GNU Compiler Collection* (GCC), cargando explícitamente el módulo `gcc/12.3.0`.

- **Directivas de Optimización y Extensiones Vectoriales:** Para la generación de los binarios ejecutables, se aplicó el nivel de optimización `-O2` junto con la macro `-DNDEBUG` para inhabilitar aserciones de depuración y maximizar el rendimiento. Adicionalmente, el sistema realiza un análisis de compatibilidad por hardware en el nodo de cómputo para activar de forma nativa las siguientes extensiones vectoriales:
 - `-mavx2` (*Advanced Vector Extensions 2*): Permite la vectorización de bucles mediante instrucciones SIMD en registros de 256 bits, procesando múltiples coordenadas de puntos 3D de forma simultánea.
 - `-mbmi2` (*Bit Manipulation Instruction Set 2*): Introduce soporte directo por hardware para instrucciones avanzadas de manipulación de bits (como PDEP y PEXT).

Asimismo, se habilitó la optimización interprocedural o en tiempo de enlazado (LTO/IPO) a través de CMake, forzando a GCC a optimizar el proyecto de forma global y permitiendo la inserción en línea (*inlining*) de funciones entre diferentes archivos fuente.

- **Soporte de Paralelismo:** Se incluyó la API **OpenMP** [?], encargada de la gestión y paralelización de hilos en memoria compartida. Se aplica en el código añadiendo directivas que definen el nivel de concurrencia en la inicialización de los bucles de búsqueda masiva sobre la estructura del Octree.

Aunque el descrito fue el entorno de ejecución real, se utilizó un entorno local de desarrollo para la programación y depuración del código fuente. Este entorno físico de soporte está constituido por una arquitectura de cómputo x86.64 basada en un procesador AMD Ryzen 7 6800H de 8 núcleos físicos y 16 hilos de ejecución concurrentes y un sistema de almacenamiento de memoria caché multinivel (L1, L2 y L3 de 512 KiB, 4 MiB y 16 MiB respectivamente).

El software se implementó sobre una capa de virtualización mediante el subsistema *Windows Subsystem for Linux* (WSL2) bajo el hipervisor de Microsoft. Sobre esta capa se desplegó una distribución del sistema operativo Linux, concretamente Ubuntu 24.04.3 LTS. Este entorno local sirvió como plataforma de validación para el diseño de los algoritmos, la verificación de la correcta aplicación de la poda y su integración con el código original antes de su posterior despliegue y compilación en los nodos de cómputo del CESGA.

3.2. Marco teórico matemático

3.2.1. Coordenadas tridimensionales

La base de este trabajo es optimizar el cálculo de distancias entre dos puntos en un espacio tridimensional. Concretamente, se trabaja con dos figuras geométricas (cubo y esfera) que envuelven los puntos de búsqueda. En ambas se hace uso de las 3 componentes cartesianas de un punto tridimensional en las comprobaciones de vecindad:

- **Kernel esférico:** Dos puntos son vecinos si la distancia d_{ij} entre el centro de búsqueda es menor al radio de búsqueda.

- Fórmula de la distancia:

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2}$$

- Existe vecindad si se cumple: $d_{ij} < R_{\text{búsqueda}}$

- **Kernel cúbico:** Dos puntos son vecinos si la distancia entre los puntos para los tres ejes dimensionales por separado es menor al radio del cubo de búsqueda:

- Para un eje $k \in \{X, Y, Z\}$, la distancia d_{kij} se define como:

$$d_{kij} = |k_i - k_j|$$

- Existe vecindad si se cumple:

$$|x_i - x_j| \leq R \wedge |y_i - y_j| \leq R \wedge |z_i - z_j| \leq R$$

Para potenciar la poda o el descarte de puntos sin tener que realizar esta comprobación, se hará uso de otros sistemas de coordenadas tridimensionales existentes aparte de los cartesianos: el sistema esférico y el cilíndrico.

Ambos son extensiones del sistema bidimensional polar. En estas coordenadas un punto se define a través de dos magnitudes: un radio R y un ángulo θ sobre el origen. Para la proyección a 3 dimensiones, existe un sistema que añade un ángulo con respecto al plano y proyecta el valor del radio bidimensional a un radio tridimensional (coordenadas esféricas) [?]. Por otra parte, se puede incluir la componente Z, que le otorga la propiedad de altura a los puntos (coordenadas cilíndricas). La conversión de un punto a estos sistemas a partir de sus componentes se hace a través de las siguientes operaciones:

- **Coordenadas esféricas**

$$R = \sqrt{x^2 + y^2 + z^2}$$

$$\varphi = \arctan(y/x)$$

$$\theta = \arccos(z/R)$$

- **Coordenadas cilíndricas**

$$R = \sqrt{x^2 + y^2}$$

$$\varphi = \arctan(y/x)$$

$$Z = z$$

En el contexto concreto de los puntos dentro del Octree, se debe tener en cuenta que para poder operar a nivel de hoja se debe trasladar el origen de las coordenadas al centro del octree antes de realizar cualquier conversión u operación.

3.2.2. Curvas de llenado de espacio

Con el fin de optimizar la localidad espacial de los datos en memoria se introducen las curvas de llenado del espacio (SFC, *Space-Filling Curves*) como herramientas de ordenación [?]. Se encargan de forzar que aquellos puntos próximos en el espacio tridimensional se almacenen de forma contigua en la zona de memoria reservada para la nube.

El proceso requiere la discretización previa del espacio continuo de la nube de puntos. Sea una nube de puntos $P \subset \mathbb{R}^3$, se define su caja contenedora o *bounding box* como la región semiabierta $B = [x_m, x_M) \times [y_m, y_M) \times [z_m, z_M)$, delimitada por sus esquinas inferiores y superiores. Fijando un nivel de subdivisión $L \in \mathbb{N}$ que determina una resolución de $N = 2^L$, se proyecta una rejilla discreta tridimensional $S_L = [0, N) \times [0, N) \times [0, N) \subset \mathbb{N}^3$, cuya cardinalidad total es de 2^{3L} celdas. Cada punto continuo $p = (x, y, z) \in P$ es trasladado, escalado y transformado en un elemento entero de la rejilla S_L mediante la aplicación de la función suelo.

En la implementación de los *Octrees* se fijó un nivel máximo de subdivisión $L = 21$. Esto permite codificar la posición tridimensional de cada celda en un único identificador entero de 64 bits (restringiendo las coordenadas discretas originales a tipos de 32 bits), minimizando la probabilidad de colisión de múltiples puntos en una misma celda sin penalizar el rendimiento del sistema. Una curva de llenado del espacio de nivel l se define formalmente como la función biyectiva:

$$C : S_l \longrightarrow [0, 2^{3l})$$

Para el propósito de evaluar el impacto de la localidad en el rendimiento de los algoritmos de búsqueda, se consideran las dos familias ya mencionadas de curvas de llenado del espacio recursivas:

Código de Morton (Curva Z)

La curva de Morton, o curva Z, mapea las coordenadas tridimensionales discretas mediante el entrelazado directo de sus bits a nivel de registros de la CPU. Para un punto discretizado en la rejilla cuyas coordenadas binarias se expresan como $p = (x_0 \dots x_{L-1}, y_0 \dots y_{L-1}, z_0 \dots z_{L-1}) \in S_L$, su código de Morton se calcula de acuerdo con la ecuación:

$$C_M(x_0 \dots x_{L-1}, y_0 \dots y_{L-1}, z_0 \dots z_{L-1}) = x_0 y_0 z_0 x_1 y_1 z_1 \dots x_{L-1} y_{L-1} z_{L-1}$$

La ventaja computacional de esta estructura radica en la velocidad y eficiencia de su cálculo. Al tratarse de un mapeo posicional directo de bits, la codificación y decodificación puede implementarse a través de operaciones lógicas de enmascaramiento (*bitmasks*) y tablas de búsqueda (LUT, *Look-Up Tables*).

Curva de Hilbert

El cálculo del código de Hilbert se realiza mediante un proceso iterativo por nivel de subdivisión en el cual, tras extraer los bits homólogos de cada

dimensión, se aplica una secuencia de transformaciones geométricas y rotaciones que dependen del estado de orientación previo y del octante evaluado.

Matemáticamente, la propiedad fundamental de las curvas de Hilbert es su continuidad en el espacio, satisfaciendo la condición de vecindad de forma que dos códigos consecutivos en el espacio unidimensional corresponden a celdas contiguas en el espacio tridimensional:

$$\|C^{-1}(i) - C^{-1}(i-1)\|_{\infty} = 1, \quad \forall i \in [1, 2^{3l-1}] \quad (3.1)$$

A nivel teórico, esta curva ofrece una localidad espacial superior a la de Morton. Sin embargo, carece de una correspondencia de bits directa y requiere un algoritmo iterativo más costoso para computar las matrices de rotación. Sin embargo, en este problema concreto debe prevalecer la calidad sobre la complejidad a la hora de calcular los reordenamientos de las nubes de puntos, ya que es una operación que solo hace falta realizar una vez. Se puede ejecutar incluso en un programa separado y cargar las codificaciones cuando se inicialice la nube.

3.3. Algoritmos y estructuras de datos de partida

En este apartado se describirán las estructuras y explicarán los algoritmos de búsqueda que se usaron como base para el trabajo y que ya fueron mencionados: el Octree lineal implementado por Viñambres et al. y sus algoritmos de búsqueda iterativos con poda a nivel de nodo [?].

3.3.1. Octree lineal

El proceso de construcción es un proceso iterativo, en el cual se parte de un nodo raíz que contiene toda la *bounding box* de la nube y divide repetidamente cada nodo terminal que contenga más de *maxLeaf* puntos, siendo este un valor paramétrico introducido por el programador. Una vez finalizada la construcción se obtiene un número N de nodos, separados en $nInternal$ (número de nodos internos) y $nExternal$ (número de nodos hoja). Todo el Octree está formado por el vector de puntos de la nube (reordenados mediante una curva de llenado y por lo tanto con una codificación asociada) junto con una serie de vectores persistentes que mapean las relaciones de descendencia entre nodos sin necesidad de usar punteros. Esta serie de vectores es la siguiente:

- `std::vector<uint32_t>offsets`

Tiene N elementos. La posición i almacena el índice de del primer hijo del nodo i . Todos los hijos de un nodo están en posiciones consecutivas. En caso de ser el nodo i un nodo hoja se cumple que `offsets[i] == 0`.

- `std::vector<std::pair<size_t, size_t>>internalRanges`

Tiene N elementos. En cada índice guarda las posiciones inicial y final de los puntos contenidos en él. Se usa para recorrer los puntos uno a uno en caso de llegar a una hoja que interseca con el kernel. Este es el punto de ineficiencia sobre el que se despliega este proyecto.

- `std::vector<Point>centers`

Cada elemento `centers[i]` contiene las coordenadas del centro geométrico del nodo i . También tiene N elementos.

- `std::vector<Vector>precomputedRadii`

Tiene L elementos siendo L la profundidad máxima encontrada en el Octree. Almacena para cada nivel de profundidad las dimensiones de los octantes.

Este formato presenta una serie de ventajas con respecto a un Octree de punteros. Se eliminan las indirecciones que provocan fallos caché y se asegura que toda la estructura esté en un grupo de bloques de memoria contiguos. Además, se conoce y se tiene acceso a los puntos contenidos por un octante, ya sea una hoja o no, y se consigue que esos puntos estén consecutivos en memoria gracias a la reordenación por SFC.

3.3.2. Algoritmos de búsqueda

A diferencia de las aproximaciones clásicas basadas en punteros, donde el recorrido del árbol requiere desreferenciar enlaces hacia nodos hijos dispersos en el montón o *heap*, el algoritmo optimizado implementado en este proyecto explota la geometría de los octantes y la contigüidad de los índices para discriminar bloques enteros de puntos.

El procedimiento general de búsqueda geométrica se rige por un esquema de recorrido recursivo sobre los nodos del árbol, evaluando la colisión espacial entre la caja del nodo actual y el volumen del kernel de consulta. El flujo algorítmico se define en los siguientes pasos:

1. **Evaluación de nodos tipo rama:** Si el nodo actual del octree es una rama (nodo interno), se computa la posición relativa de su volumen geométrico respecto a las fronteras del kernel de búsqueda:
 - a) **Inclusión completa:** Si el octante se encuentra totalmente contenido en el interior del kernel, se activa el mecanismo de rangos internos (*internal ranges*). Esto permite indexar y volcar de manera secuencial todos los puntos contenidos en dicho subárbol sin necesidad de descender más niveles ni evaluar individualmente sus elementos.
 - b) **Exclusión completa:** Si el octante se halla completamente fuera de los límites del kernel, se procede a la poda (*pruning*) inmediata de la rama, abortando esa vía de exploración.
 - c) **Intersección parcial:** Si el octante interseca con las fronteras del kernel pero no está contenido estrictamente en él, se continúa el recorrido recursivo descendiendo hacia sus nodos hijos.
2. **Evaluación de nodos tipo hoja:** Al alcanzar un nodo hoja que interseca parcialmente con el kernel, el algoritmo desciende al nivel atómico del dato, validando punto a punto mediante criterios geométricos si sus coordenadas pertenecen o no al interior del volumen de consulta.

3. **Retorno de resultados:** Tras concluir el recorrido, se consolida y devuelve la colección de índices de los puntos que se encuentran estrictamente contenidos en el espacio delimitado por el kernel.

Para la materialización y almacenamiento de estos índices resultantes, se han desarrollado dos variantes algorítmicas diferenciadas por su estrategia de gestión de memoria y estructuración de datos: **neighborsPrune** y **neighborsStruct**.

Algoritmo **neighborsPrune**

Esta variante implementa una estrategia de inserción directa y plana. Cuando el algoritmo identifica puntos válidos en el interior del kernel (ya sea mediante la evaluación individual en hojas o por la inclusión completa de una rama), añade cada índice de punto vecino como un elemento independiente y separando dentro de un vector dinámico estándar (`std::vector<size_t>`). Si bien el coste de inserción es secuencial, esta aproximación puede resultar innecesaria en nubes densas, donde se insertan numerosos puntos consecutivos.

Algoritmo **neighborsStruct**

Con el objetivo de mitigar el coste de inserción masiva de elementos individuales, la variante **neighborsStruct** aprovecha la contigüidad física de los datos reordenados en memoria. En lugar de almacenar elementos aislados, define una estructura de rangos indexados mediante el tipo:

```
std::vector<std::pair<size_t, size_t>> ranges;
```

Bajo este esquema, cuando una secuencia de puntos consecutivos en memoria pertenece de forma unánime al kernel de consulta (caso seguro al incluir ramas totalmente contenidas), el algoritmo compacta la información realizando únicamente dos operaciones de inserción por bloque: el índice de la posición inicial y el índice de la posición final del rango. Esta técnica reduce drásticamente el espacio de almacenamiento temporal, minimiza las escrituras en memoria caché, optimizando la velocidad del algoritmo.

Mediciones temporales

Para evaluar de manera rigurosa el rendimiento temporal de las estructuras y algoritmos propuestos, en este estudio se diseñó e implementó un sistema de medición integrado en la infraestructura del proyecto. Las mediciones temporales se encapsularon en una clase denominada **TimeWatcher**, la cual abstrae y centraliza la captura de métricas temporales a través de una interfaz basada en los métodos **start()** y **stop()**.

La infraestructura interna de esta clase hace uso de la biblioteca estándar de C++ `<chrono>` [?]. En concreto, el inicio y el fin de cada intervalo de evaluación se gestionan mediante punteros inteligentes únicos (`std::unique_ptr`), los cuales aseguran una gestión de memoria eficiente y un *overhead* mínimo.

La función usada para marcar los instantes temporales es `high_resolution_clock`. Este reloj es el más preciso de la biblioteca `<chrono>`, llegando a definir métricas de nanosegundos.

Con el objetivo de mantener una consistencia en la fase experimental de mi estudio, se usarán exactamente estas mediciones temporales ya implementadas e integradas en la arquitectura del programa.

3.4. Conjuntos de datos empleados

Las nubes de puntos 3D que fueron usadas para evaluar el rendimiento del algoritmo de poda a nivel de hoja vienen descritos a continuación, en el Cuadro ???. Las nubes están almacenadas en archivos binarios estándar con extensión `.LAS`. Este formato contiene los datos espaciales tridimensionales aparte de campos opcionales con más propiedades de los puntos, como intensidad, color o clasificaciones de terreno.

La selección de nubes de puntos se realizó tratando de buscar la máxima variedad posible: se mezclan conjuntos de datos de baja y alta densidad, así como nubes extraídas con sensores LiDAR tanto aéreos como terrestres y por último variedad en el número total de puntos contenidos.

Nube	nPuntos	Dataset	Descripción
<i>bildstein_station1_xyz intensity_rgb</i>	29.6 M	Semantic3D [?]	Escaneo LiDAR terrestre y urbano desde una posición estática. Nubes muy densas.
<i>sg27-intensity_rgb</i>	73.9 M		
<i>5085_54320.las</i>	14.4 M	DALES [?]	LiDAR aéreo obtenido desde aeronaves. El conjunto se divide en celdas disjuntas.
<i>5095_54440.las</i>	11.7 M		
<i>Lille_0</i>	10.0 M	Paris-Lille 3D [?]	Sensor LiDAR de tipo Mobile Laser Scanning (MLS) instalado sobre un vehículo que mapea dos calles urbanas en París.
<i>Paris_Luxembourg_6</i>	10.0 M		
<i>PNOA_2024_PNR 489-4672_NPC01</i>	18.5 M	PNOA-LiDAR [?]	Cobertura del territorio español mediante LiDAR aéreo. Presenta una densidad baja, cubriendo grandes extensiones.
<i>Mar18_train</i>	59.4 M	Hessigheim 3D [?]	Dataset aéreo de alta resolución capturado mediante UAV (<i>Unmanned Aerial Vehicle</i>) sobre el pueblo de Hessigheim.
<i>Mar18_test</i>	51.7 M		

Cuadro 3.2: Nubes de puntos utilizadas en las evaluaciones experimentales.

Capítulo 4

Metodología

En este capítulo se expone detalladamente el proceso de diseño, implementación y optimización incremental de los nuevos algoritmos de poda geométrica a nivel de nodo hoja sobre la estructura del Octree lineal de partida. El objetivo metodológico fundamental de este estudio es romper el carácter indivisible de los nodos hoja tradicionales, acotando el espacio de búsqueda lineal mediante la aplicación de criterios analíticos basados en proyecciones cartesianas, esféricas y cilíndricas.

A diferencia de un desarrollo estático, la metodología de este proyecto se ha concebido como un proceso iterativo guiado por el rendimiento. A lo largo de las siguientes secciones se describirán las técnicas utilizadas a lo largo de la evolución del software. Se partió de un modelo teórico inicial enfocado en la máxima restricción geométrica y el ahorro de memoria, pasando por el análisis de cuellos de botella computacionales hasta converger en una solución de alta localidad espacial y bajo coste de computación.

4.1. Fundamentos geométricos de la poda en nodos hoja

En esta sección se explica cómo se realiza exactamente la poda para descartar puntos dentro de hojas conflictivas. Como ya se ha visto anteriormente, es sencillo conseguir los valores de coordenadas esféricas o cilíndricas a partir de coordenadas cartesianas tridimensionales. Para cada propiedad que las define, se puede delimitar unos valores máximo y mínimo que un punto puede tener para que, dado un kernel de búsqueda definido, pueda estar dentro de ese kernel de vecindad, de forma que si no tiene un valor fuera de ese intervalo se descartará automáticamente. Lo interesante de esta cuestión es no tener que calcular esos valores uno a uno, eso es exactamente lo que hacen los algoritmos originales. Lo que convierte este cálculo en un algoritmo eficiente es ordenar los puntos, de forma independiente en cada hoja, según cada una de esas claves en la inicialización de la estructura. De esta forma, basta con encontrar los puntos que suponen el límite inferior y superior del intervalo y se tiene automáticamente el conjunto de puntos a evaluar, descartando el resto. La eficiencia de esta técnica depende de dos factores fundamentales: la efectividad directa de

la poda, es decir, el porcentaje de puntos descartados, que debe ser lo mayor posible; y la reducción de la sobrecarga en el cálculo, debiendo evitar que el coste de definir los índices sobre los que buscar no supere el coste de rechazar los puntos descartados uno a uno, tal y como se estaba haciendo.

A continuación se detalla como se definen los valores máximos y mínimos para cada propiedad posible del punto. Una consideración a tener es que este proceso no es único de las coordenadas polares, se puede tomar la misma técnica con los valores de los ejes X, Y, Z. De hecho, es *a priori* una buena opción ya que se evitan las operaciones de transformación de sistema de coordenadas, por lo que es una estrategia que se tendrá en cuenta. En la Figura ?? se puede ver una simulación gráfica de cómo se realiza la poda en el espacio de 3 dimensiones, en este caso usando el eje X para la poda. El cubo negro representa la hoja del Octree mientras que el azul representa el kernel de búsqueda cúbico, con el punto de búsqueda en su centro. La zona cortada por el hiperplano que queda en el interior de la hoja es la única zona donde puede haber vecinos, sea la distribución que sea.

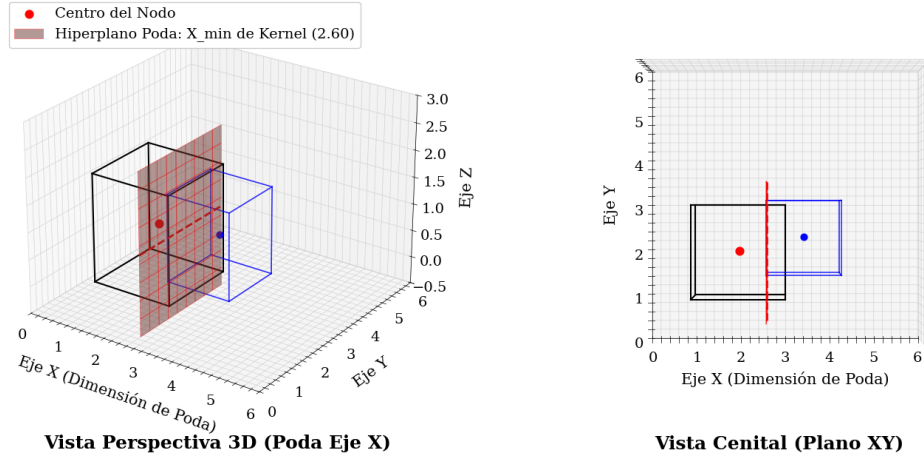


Figura 4.1: Funcionamiento de la poda en un espacio tridimensional usando ejes cartesianos.

El cálculo matemático de los límites de búsqueda es directo si se traslada el origen al centro de la hoja del Octree. Dado un punto de búsqueda $Q = (q_x, q_y, q_z)$ y un radio de búsqueda r_k para cada eje $k \in \{X, Y, Z\}$ (ya sea una esfera o un cubo), un punto cualquiera $P = (p_x, p_y, p_z)$ perteneciente a dicha hoja se considera un candidato potencial a vecino si y solo si sus coordenadas satisfacen la siguiente condición para el componente $k \in \{X, Y, Z\}$ que está siendo estudiado:

$$p_k \in [q_k - r_k, \quad q_k + r_k]$$

Lo cual equivale a la restricción de vecindario por componentes dada por el valor absoluto:

$$|p_k - q_k| \leq r_k$$

4.1. FUNDAMENTOS GEOMÉTRICOS DE LA PODA EN NODOS HOJA21

Aquí se debe realizar un apunte. La restricción real para que un punto de búsqueda $Q = (q_x, q_y, q_z)$ sea vecino es la agregación de la restricción anterior para todas sus componentes:

$$\forall k \in \{X, Y, Z\}, \quad p_k \in [q_k - r_k, \quad q_k + r_k]$$

Sin embargo, de esta forma no se pueden hacer ordenaciones y habría que hacer intersecciones de rangos para hallar los puntos candidatos. Esto eliminaría el objetivo de la optimización, evitar tener que comprobar puntos uno a uno, además de eliminar la localidad y suponer una sobrecarga demasiado grande. Por estos motivos se trabajará con las propiedades de los puntos de forma separada para la extracción de rangos.

De forma análoga, es posible determinar analíticamente los límites inferiores y superiores para las variables de los sistemas de coordenadas alternativos. Dado un punto de búsqueda $Q = (q_x, q_y, q_z)$ y un radio de consulta r , se debe realizar una consideración previa cuando se emplean entornos volumétricos ortogonales: en el caso del kernel cúbico, para garantizar que ningún candidato potencial quede excluido y no sobreestimar las condiciones de vecindad, se toma como radio efectivo el de la esfera mínima circunscrita que lo envuelve completamente, el cual se define formalmente como $r_{\text{efectivo}} = r\sqrt{3}$.

En el marco del sistema de **coordenadas esféricas**, la evaluación del ángulo azimutal φ (cuyo dominio se restringe al intervalo $[-\pi, \pi)$) se estructura a partir de la proyección del punto de búsqueda en el plano xy . Se define la distancia de la *query* al origen como $\rho_q = \sqrt{q_x^2 + q_y^2}$. El ángulo azimutal del punto de búsqueda se calcula mediante la función arcotangente de dos parámetros como $\varphi_q = \text{atan2}(q_y, q_x)$.

Geométricamente, la máxima desviación angular posible que puede tener un vecino a una distancia horizontal ρ_q viene acotada por las tangentes a la esfera de búsqueda. Por lo tanto, el intervalo de tolerancia angular dentro del cual debe encontrarse un punto para optar a ser vecino se define como:

$$\Phi = [\varphi_q - \Delta\varphi, \quad \varphi_q + \Delta\varphi]$$

Donde el semiángulo de apertura $\Delta\varphi$ se deduce de la relación trigonométrica:

$$\Delta\varphi = \arcsin\left(\frac{r}{\rho_q}\right)$$

Continuando con el sistema de **coordenadas esféricas**, la segunda componente angular a evaluar corresponde al ángulo de coaltitud o ángulo polar θ , cuyo dominio físico se restringe al intervalo $[0, \pi]$. Este ángulo describe la desviación respecto al eje de ordenadas Z y se calcula analíticamente a partir de la distancia euclídea tridimensional al origen, definida como $r_q = \sqrt{q_x^2 + q_y^2 + q_z^2}$, mediante la relación:

$$\theta_q = \arccos\left(\frac{q_z}{r_q}\right)$$

De manera homóloga al análisis azimutal, la dispersión angular máxima para los candidatos a vecinos en esta dimensión se encuentra acotada por el

contorno de la esfera de búsqueda de radio r . Por consiguiente, el intervalo de aceptación para el ángulo polar se establece como:

$$\Theta = [\theta_q - \Delta\theta, \quad \theta_q + \Delta\theta]$$

Donde la apertura angular crítica $\Delta\theta$ se deriva directamente como:

$$\Delta\theta = \arcsin\left(\frac{r}{r_q}\right)$$

No obstante, existe una condición geométrica singular para estas dos componentes angulares que hace automáticamente nula la poda y permite mitigar el coste computacional al eludir el cálculo de funciones trigonométricas inversas, de alto coste en CPU. La estrategia de poda es estéril si se satisface la relación:

$$r \geq r_q \quad (\text{o } r \geq \rho_q \text{ en el plano } xy)$$

La naturaleza de esta restricción radica en que, si el radio de consulta es mayor o igual a la distancia del punto de búsqueda al origen, el volumen del kernel contiene de forma implícita dicho origen. En consecuencia, el espacio de búsqueda abarca todas las direcciones del espacio y por lo tanto todo el dominio de valores angulares, imposibilitando cualquier restricción angular efectiva.

Por último, la cota asociada a la componente radial tridimensional r_p de cualquier punto candidato P para el filtrado en esféricas es directa, obligando al punto a posicionarse dentro del rango de distancias determinado por:

$$R_{\text{esférico}} = [r_q - r, \quad r_q + r]$$

En contraposición, cuando el análisis se reduce a un sistema de **coordenadas cilíndricas**, la componente radial se evalúa de forma bidimensional. La distancia proyectada de la *query* se define como $\rho_q = \sqrt{q_x^2 + q_y^2}$, y el criterio de vecindad para la magnitud del radio cilíndrico ρ_p de los puntos de la hoja se simplifica al intervalo:

$$R_{\text{cilíndrico}} = [\rho_q - r, \quad \rho_q + r]$$

En la Figura ?? se ilustra un ejemplo de mecanismo de poda en el espacio tridimensional, empleando en este caso el ángulo azimutal φ . Como se observa, el algoritmo traza dos hiperplanos orientados según los límites angulares calculados, descartando inmediatamente cualquier punto cuyos límites geométricos queden completamente fuera del sector angular Φ .

4.2. PRIMERA FASE DE IMPLEMENTACIÓN: SELECTORES DE RANGO POLARES Y VECTOR

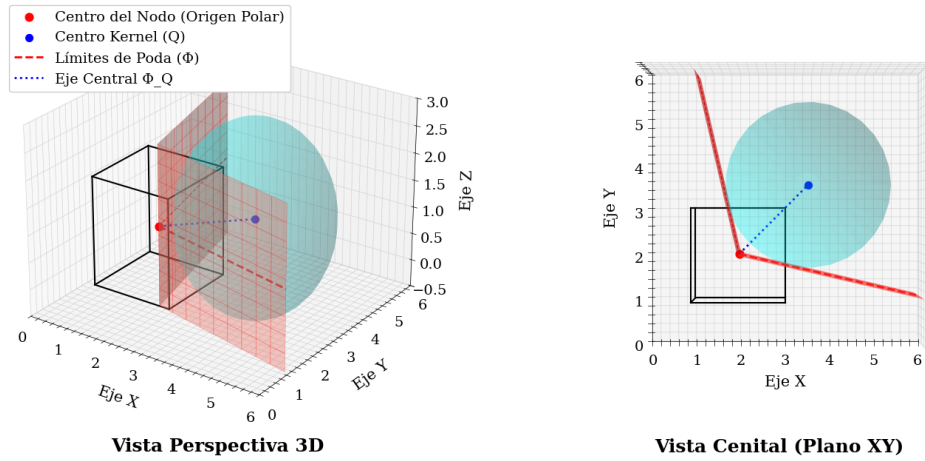


Figura 4.2: Funcionamiento de la poda en un espacio tridimensional usando coordenadas esféricas/cilíndricas.

4.2. Primera Fase de Implementación: Selectores de rango polares y vectores de permutaciones

Las decisiones de diseño iniciales se caracterizaron por buscar un equilibrio entre la minimización de la sobrecarga en memoria introducida y en maximizar la cuota de puntos podados. Un trabajo inicial que hubo que realizar fue exponer una serie de funciones en la clase del Octree lineal para poder operar a nivel de hoja con comodidad. Estas funciones dan acceso a una serie de vectores que contienen los centros de las hojas y pares de índices que indican el inicio y el final de los puntos de la hoja en la estructura original del Octree, así como una función que devuelve el número total de hojas. Una vez integradas estas funciones preliminares, la primera parte de la implementación es construir el reordenamiento de puntos a nivel de hoja. Esto se hace a la hora de inicializar la estructura y las operaciones solo se llevan a cabo una vez al cargar la nube de puntos, por lo que la eficiencia temporal no es una gran preocupación. Sí lo puede ser, sin embargo, el consumo de memoria. Por eso la primera aproximación fue la de crear vectores de permutaciones. La utilidad de estos es almacenar, dentro de una hoja, el índice global de los puntos en orden creciente siguiendo una clave concreta de las analizadas en la sección anterior. Para ordenar cada vector se usó la función `std::sort [?]`, con complejidad temporal $\mathcal{O}(N \log(N))$. Así, para acceder, por ejemplo, al punto con el radio esférico menor de la hoja i habría que acceder a `points[permsRadius[i][0]]` siendo `points` el array de puntos originales. El beneficio de este método es que cada elemento es un `size_t` y ocupa 8 bytes frente a los 32 bytes que ocupa un objeto de la clase `Point`. Por otro lado, se diseñó la técnica de selección de rango. Esta funcionalidad está pensada para ser ejecutada una vez por cada ocasión en la que se llegue a una hoja conflictiva durante una búsqueda, por lo que se ejecutará miles de veces por ejecución y será trascendental buscar la máxima eficiencia temporal, a la vez que se debe conseguir una poda lo mejor posible. Este equilibrio a

buscar es el factor más decisivo a la hora de enfocar el diseño de esta función. Este módulo debe devolver los rangos mínimo y máximo para examinar de la hoja que se está evaluando. En ella se hacen los cálculos matemáticos descritos en la anterior sección para conseguir los límites numéricos y a continuación se lleva a cabo una búsqueda binaria sobre los puntos reordenados de la hoja para transformar los valores en el intervalo de índices correspondiente, por lo que la complejidad del selector es $\mathcal{O}(\log_2 N)$, aparte de la sobrecarga de complejidad constante introducida por los cálculos matemáticos.

Se implementaron de salida dos modos de reordenamiento independientes, uno usando las características de un punto según coordenadas esféricas y otro según coordenadas cilíndricas. Se hizo de esta manera porque no tiene sentido mezclar claves de ambos sistemas en una única búsqueda: Las podas que se pueden realizar con coordenadas esféricas son por definición más restrictivas que las que se pueden hacer con el sistema cilíndrico. La poda por el ángulo φ es directamente la misma. La poda que el ángulo θ lleva a cabo se define por la posición en el eje Z del punto de búsqueda, pero es más estricta que podar directamente en base al valor de Z (la ventaja del sistema cilíndrico aquí es que puede podar casos en los que el kernel cubra el centro). Por último, el radio tridimensional es más estricto que el radio en el plano XY. En conclusión, las coordenadas esféricas consiguen un rango más estricto mientras que las coordenadas cilíndricas son algo menos costosas computacionalmente. Esto hace que no tenga sentido mezclar los sistemas en las búsquedas, y proponen una comparativa interesante a estudiar entre los dos modos. Se añadió como parámetro de entrada el parámetro `--local-reorder`, que podía tomar uno o varios valores de entre los siguientes: `[none,spherical,cylindrical]` Una vez tomada la decisión de separar las reordenaciones esféricas y cilíndricas se definieron los detalles de la estrategia en la selección de rango, que fue la siguiente:

1. Se investigan las tres posibles reordenaciones para el sistema de coordenadas elegido. Para cada clave:
 - a) Se calculan los valores límite tal y como se expuso anteriormente.
 - b) Se traducen los valores a índices mediante la búsqueda binaria.
2. Al finalizar el bucle, se comprueba qué clave deja un rango más pequeño a examinar.
3. Se devuelve un código que se asocia con el reordenamiento elegido, los índices mínimo y máximo y el vector de permutaciones correspondiente.

4.3. Análisis de Cuellos de Botella y Rediseño del Sistema

Una vez terminada la implementación e integración de la versión base de la poda, se llevaron a cabo unas pruebas preliminares para poder tener una primera referencia de su desempeño. Los resultados fueron negativos para diferentes tipos de nubes, por lo que hubo que pasar por un proceso de análisis

4.3. ANÁLISIS DE CUELLOS DE BOTELLA Y REDISEÑO DEL SISTEMA25

exhaustivo para encontrar las causas y soluciones del mal rendimiento de la implementación de las podas con respecto a los algoritmos base. Para conocer la efectividad de la poda en sí se introdujo el modo de debug en el programa, con el que se exportan a un archivo información de cada hoja conflictiva a la que se llega. Se imprime qué modo de reordenamiento se está usando (cilíndrico o esférico), qué clave concreta fue mejor y por tanto se usó en esa hoja, el número de puntos de la hoja y el número de puntos restantes después de la poda. Con esta información se llevó a cabo un estudio de la efectividad de la poda sobre un pequeño conjunto de nubes que dio lugar a los resultados mostrados en el Cuadro ???. También se analizó el éxito de cada clave de reordenamiento, cuyos resultados se recogen en las Figuras ?? y ??.

Reordenamiento	Radio (r)	Paris-Lille		sg_27_station		Lille_0	
		Cubo	Esfera	Cubo	Esfera	Cubo	Esfera
Cilíndrico	0.5	13.8 %	37.0 %	39.2 %	51.9 %	5.2 %	28.2 %
	1.0	8.5 %	28.8 %	6.4 %	33.3 %	6.1 %	28.4 %
	2.0	7.0 %	26.0 %	3.6 %	31.7 %	12.4 %	33.5 %
	3.0	6.7 %	23.9 %	2.4 %	32.2 %	17.2 %	39.1 %
Esférico	0.5	9.3 %	38.7 %	22.3 %	48.6 %	2.8 %	32.3 %
	1.0	3.9 %	32.6 %	6.2 %	36.2 %	3.8 %	32.4 %
	2.0	2.6 %	31.4 %	3.5 %	33.8 %	9.2 %	35.5 %
	3.0	1.4 %	30.8 %	2.3 %	34.2 %	13.8 %	40.1 %

Cuadro 4.1: Porcentaje de puntos podados según el tipo de reordenamiento, geometría del kernel y radio de búsqueda (r) para las nubes analizadas.

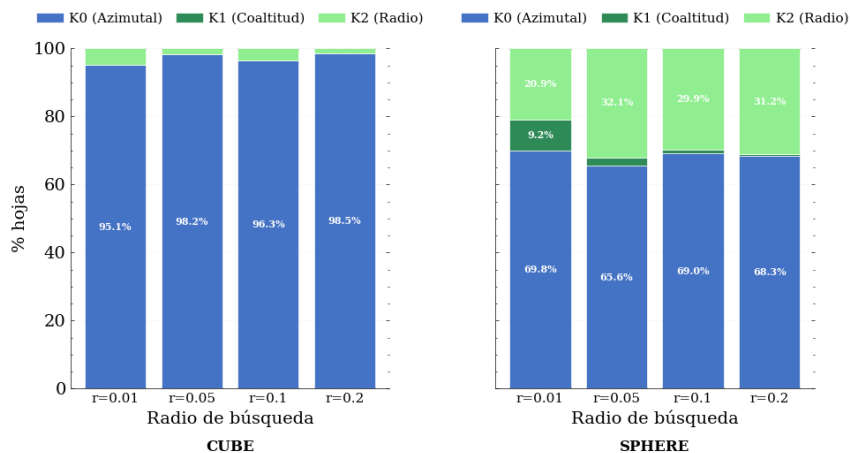


Figura 4.3: Distribución de claves óptimas por hoja en modo esférico sobre la nube sg27_station_8

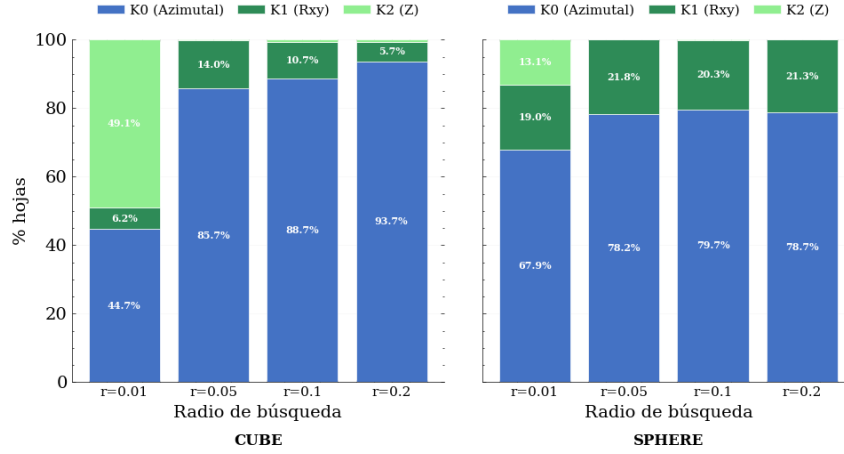


Figura 4.4: Distribución de claves óptimas por hoja en modo cilíndrico sobre la nube sg27_station_8

De estos datos se pueden extraer varios aprendizajes. En primer lugar, las podas con kernel cúbico tienen una efectividad prácticamente nula debido a la sobreestimación que hay que realizar, lo cual probablemente provoque que para la mayoría de hojas el kernel cubra el centro o incluso la hoja entera. Ese porcentaje que se ahorra, de entre el 5 y el 10 % de las hojas, no compensa para nada la sobrecarga introducida y el gasto en memoria. También se aprecia un comportamiento a tener en cuenta de aquí en adelante, y es que cuánto más pequeño sea el radio, más efectiva será la poda de forma general. Esto ocurre así porque la sección angular que cubre una esfera, estando a la misma distancia siempre del centro, es proporcional al tamaño de esta, y esferas pequeñas se pueden encerrar en regiones muy estrechas. Por último, en las Figuras ?? y ?? hay un patrón muy claro. La clave que mejor funciona con mucha diferencia es el ordenamiento por ángulo φ , seguido de lejos por la coordenada Z .

Este estado de conocimiento dio pie a una serie de optimizaciones y rediseños que fueron aplicados sobre los algoritmos.

4.4. Segunda Fase de Implementación: Refactorización y Simplificación de Modelos

Tras analizar los resultados preliminares, se tomaron una serie de cambios en el enfoque de las podas. Los principales problemas a arreglar eran la ineffectividad con el kernel cúbico y la excesiva sobrecarga en proporción a la magnitud de hojas podadas.

Dada la dominancia del ángulo φ para los reordenamientos, se decidió usar solo esa clave de aquí en adelante. Esta decisión tiene una desventaja, y es que se quedarán sin podar hojas que quizás con el ángulo de coaltitud o el radio sí descartarían puntos. Pero las ventajas son un gran ahorro en memoria, teniendo en cuenta que también se eliminan los dos modos separados de coordenadas esféricas y cilíndricas, y un gran ahorro en la sobrecarga de la selección de rangos. El funcionamiento de la selección de rango en las búsquedas ahora

4.5. CONSTRUCCIÓN DE RÉPLICAS CONTIGUAS E INTEGRACIÓN EN EL FLUJO DE BÚSQUEDA

simplemente consiste en calcular los valores límites angulares en el dominio $[-\pi, \pi)$, realizar la búsqueda binaria para encontrar el rango de puntos en la hoja y devolver el rango junto con el vector de permutaciones necesario para efectuar la búsqueda. Cabe destacar, eso sí, un condicional implementado a mayores. El dominio de valores es circular, es decir, que un kernel puede soportar un rango del tipo $[x, \pi), [-\pi, y]$. Se incluyó esta posibilidad devolviendo dos pares de índices e incluyendo un nuevo condicional en los algoritmos de búsqueda, que recorren el segundo intervalo de puntos en caso de ser necesario.

En este punto se tomó la decisión de implementar otro tipo de reordenamiento, de forma que hubiese dos modos independientes de nuevo. En este caso, se usarían los valores de los ejes de coordenadas X, Y, Z para hacer los reordenamientos a nivel de hoja, de forma que habría 3 posibles claves, al igual que en el primer algoritmo propuesto. Sin embargo, en este caso hay una serie de ventajas que hacen esta implementación mucho más atractiva. Para empezar, los valores cartesianos ya están definidos en los puntos, no hay que hacer cálculos a mayores para transformar las coordenadas y posteriormente ordenar los puntos. Además, el cálculo de límites es extremadamente rápido. Tal y como se detalló en la sección 4.1, no es más que una suma/resta. A mayores, se espera que este formato arregle el problema presente en los resultados de poda obtenidos de la dependencia al tamaño del radio. Los ejes cartesianos pueden podar cualquier sección de la hoja, mientras que el efecto del ángulo azimutal se anula cuando el kernel sobrepasa el centro de la hoja. Por último, hay una nueva optimización sobre la selección de rango posible para este caso. El cálculo de la región entre los valores límites es trivial y nada costosa, por lo que se puede usar como referencia para determinar cuál es el eje que más puntos desecha y se elimina la necesidad de ejecutar las búsquedas binarias por partida triple. Dado que no se asegura una distribución uniforme de los puntos, este método no asegura que se esté escogiendo la clave óptima siempre, pero es una aproximación muy eficiente temporalmente y no se espera una pérdida significativa en efectividad de poda.

Por tanto, se llevó a cabo la refactorización, unión y simplificación del algoritmo de poda basado en coordenadas esféricas/cilíndricas (modo de reordenamiento polar de aquí en adelante) y se implementó el modo basado en coordenadas ordenadas (modo cartesiano de aquí en adelante). La implementación de las funciones de selección de rango y algoritmos de búsqueda se hizo en funciones separadas para aumentar el desacoplamiento y evitar saltos condicionales en tiempos de ejecución.

4.5. Construcción de réplicas contiguas e integración en el flujo de búsqueda del Octree

En paralelo a la implementación de los nuevos modos polar y cartesiano, se analizó y diseñó otra optimización temporal, basada esta vez en la localidad de memoria durante el bucle de búsqueda. Hasta ahora, las reordenaciones se estaban haciendo mediante vectores de permutaciones, lo que provocaba accesos a memoria extra en relación al algoritmo base. En las Figuras ?? y ?? vemos

una comparación entre el bucle base y el modificado para poder recorrer el intervalo podado mediante indirecciones para el algoritmo `neighborsPrune`:

Require:

leafStart, leafEnd, points, k

Ensure: *ptsInside*

```

1: for i = leafStart to leafEnd - 1
  do
2:   if k.isInside(points[i]) then
3:     ptsInside.push_back(i)
4:   end if
5: end for

```

Figura 4.5: Búsqueda base (Acceso secuencial).

Require:

rangeStart, rangeEnd, points, perms, k

Ensure: *ptsInside*

```

1: for i = rangeStart to rangeEnd - 1
  do
2:   if k.isInside(points[perms[i]])
    then
3:     ptsInside.push_back(perms[i])
4:   end if
5: end for

```

Figura 4.6: Búsqueda reordenada (Acceso indexado).

El objetivo es poder aplicar el bucle ?? a la vez que las podas a nivel de hoja. Para conseguirlo, es necesario hacer una réplica del vector de puntos de forma que cada hoja empiece en el mismo índice, para mantener la coherencia con la parte exterior del algoritmo, pero estando los puntos reordenados a nivel de hoja. Esta decisión introduce un gasto mayor en memoria, pero se asume teniendo en cuenta que el objetivo principal del trabajo es encontrar eficiencia temporal y que hasta ahora los resultados no habían sido positivos, además de que con la simplificación de los algoritmos de selección de rangos ya se hizo un ahorro de memoria sobre la versión inicialmente planteada. Concretamente, el tamaño de cada vector contiguo ocupa $64 * N$ bytes (siendo N el número total de puntos), es decir, exactamente el mismo tamaño que el vector original. De esta forma, después de la construcción de la estructura, se intercambia la creación de los vectores persistentes de permutaciones por duplicaciones de `points` con reordenamientos incluidos a nivel de hoja. Este enfoque supone un ahorro espacial notable en el caso del reordenamiento polar. Solo existe un reordenamiento, además de que después de su creación, el vector original ya no es necesario, por lo que se puede liberar. No ocurre lo mismo con las podas cartesianas, donde se asume una copia triplicada de los puntos.

En el caso del modo polar, este cambio funcionó de manera correcta. El formato del bucle de búsqueda es el mismo. Los índices guardados en el almacén de vecinos `ptsInside` hacen referencia al vector de reordenamiento polar `polarPoints`. La única diferencia es que en este modo se incluye el bucle condicional sobre el segundo rango que se recorrerá en caso de existir, tal y como se expuso en la sección 4.4.

Sin embargo, en el modo cartesiano la búsqueda no es tan sencilla. Se están mezclando tres vectores de reordenamiento diferentes en diferentes hojas de la misma búsqueda. Por tanto hay que recuperar un puntero al inicio de la hoja de uno de ellos dependiendo de cuál produjese la mejor poda. Aparte de esa mínima sobrecarga, hay un problema mayor a la hora de incluir los índices en

4.5. CONSTRUCCIÓN DE RÉPLICAS CONTIGUAS E INTEGRACIÓN EN EL FLUJO DE BÚSQUEDA

el almacén de vecinos. No se pueden añadir mediante la forma `push.back(i)` porque una vez terminada la búsqueda, a la hora de acceder y reconstruir los vecinos, sería imposible saber a qué vector apunta cada índice.

La solución que se adoptó fue usar el campo `id` existente en la estructura `Point`. En ella, a la hora de construir los vectores reordenados, se incluyeron las posiciones originales de cada punto. De esta forma, durante el bucle se insertan los puntos accediendo a la función `id()` del punto, una función *getter* del campo. La ineficiencia de este acceso extra existe, pero teniendo en cuenta que es una función *inline* y que el objeto `Point` fue accedido justo antes y debería estar en caché, es la mejor solución posible ante el problema planteado.

Esta explicación, sin embargo, solo aplica al algoritmo `neighborsPrune`. El algoritmo `neighborsStruct` incluye una forma de almacenado de vecinos basada en pares de índices, como se aprecia en la siguiente Figura ??.

Require: *leafStart, leafEnd, points, k*

Ensure: *ptsInside*

```

1: rangeStart  $\leftarrow$  leafStart
2: for i = leafStart to leafEnd - 1 do
3:   if  $\neg k.isInside(points[i])$  then
4:     if rangeStart < i then
5:       result.addRange(rangeStart, i)
6:     end if
7:     rangeStart  $\leftarrow$  i + 1
8:   end if
9: end for

```

Figura 4.7: Búsqueda base para el algoritmo `neighborsStruct`.

Nuevamente, el modo polar aplica exactamente la misma lógica que las búsquedas originales, por lo que la adaptación a este algoritmo es trivial. Por otra parte, hasta donde se analizó, no es posible la adaptación del modo cartesiano. Los rangos en un vector de reordenamiento concreto se corresponden con puntos aleatoriamente distribuidos en el vector `Points`, por lo que es imposible hacer una inserción de vecinos mediante rangos.

Para terminar el capítulo de implementación, cabe mencionar que se introdujo un parámetro personalizable `threshold` al programa. Su función es impedir que la sobrecarga de selecciones de rango sea mayor al tiempo ahorrado en el bucle de búsqueda. Por tanto, cualquier hoja con $N_{puntos} < threshold$ será recorrida de forma secuencial.

Capítulo 5

Pruebas

En este capítulo se expondrán los experimentos ejecutados en los nodos de computación del CESGA y los resultados obtenidos. El experimento se divide en dos secciones: el análisis principal, el de tiempos de ejecución, y un análisis complementario de escalabilidad de paralelización. Los experimentos de tiempos de ejecución se dividen a su vez en búsquedas de subconjuntos con centros aleatorios y en búsquedas sobre nubes completas.

5.1. Plan de Verificación y Corrección Global

Como paso previo al análisis de rendimiento, se diseñó un script de pruebas unitarias automatizadas para verificar la fidelidad de los nuevos algoritmos orientados a la poda en nodos hoja. Para cada una de las configuraciones evaluadas en este capítulo, se contrastaron los resultados de vecindad devueltos por las variantes optimizadas frente a los resultados reportados por la implementación base del Octree lineal clásico. La verificación produjo una coincidencia del 100 %, por tanto se pasó al estudio de los resultados temporales, con la garantía de que en las podas se descartan únicamente regiones del espacio geométrico disjuntas al kernel.

Adicionalmente, se usó el modo de debug implementado para tener una medida de la efectividad teórica de las podas, más allá de saber si son eficientes temporalmente o no. En la Figura ?? se recoge el porcentaje puntos analizados después de calcular la poda y el porcentaje de puntos que resultaron ser vecinos en relación al número de puntos totales de cada hoja para la nube de puntos de *Lille_0*. Los resultados a lo largo de las diferentes nubes fueron muy consistentes, siendo este dataset lo suficientemente representativo. Para obtener estos resultados se usó una muestra de 80 búsquedas con centros aleatorios. El caso ideal es que el número de puntos analizados después de la poda y el número de puntos vecinos sea el mismo. El tamaño de los kernels se muestra representado como la media de la relación entre el radio y el tamaño de las hojas que se podan, de forma que se pueda tener una visión más independiente al dataset de cómo funciona la poda con diferentes tamaños de búsqueda.

Los porcentajes mostrados indican un comportamiento claro: el modo polar poda de manera aceptable en las búsquedas con kernel esférico, pero apenas

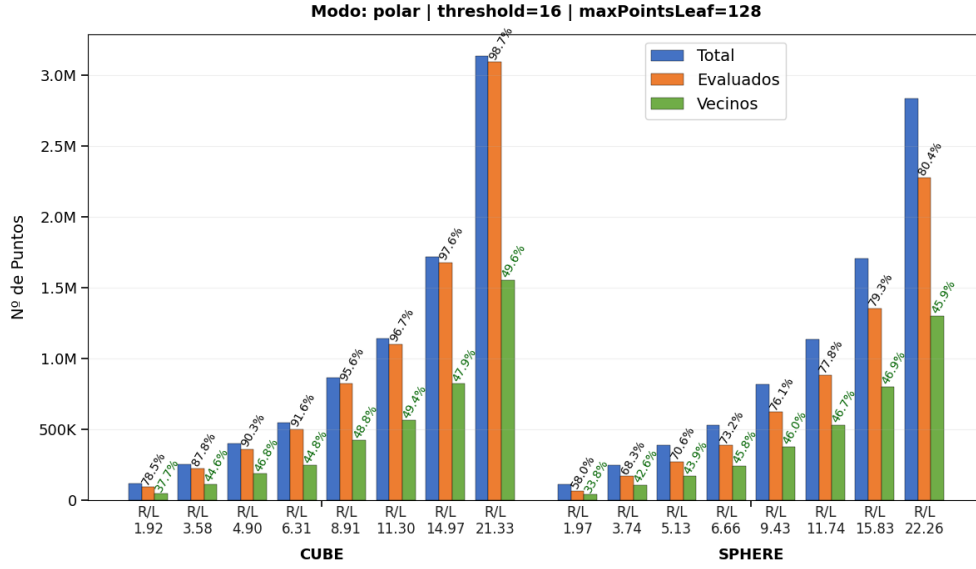
poda en búsquedas cúbicas. Por el contrario, el modo cartesiano realiza podas que están cerca de ser ideales para geometría cúbica, en cambio descarta muy pocos puntos en búsquedas esféricas. En la poda polar hay otra tendencia clara observable, y es que a medida que el radio es más pequeño en relación a la magnitud de los octantes, se descarta un mayor porcentaje de puntos y se reduce la diferencia con el porcentaje de vecinos. Por otra parte, en el reordenamiento cartesiano sobre kernel esférico ocurre lo contrario, se abre esa distancia a medida que aumenta el radio de búsqueda. Aún así, esta degradación es muy leve y continúan siendo podas muy efectivas (no se supera el 25 % de diferencia entre puntos recorridos y aceptados en ningún caso).

5.2. Rendimiento de búsquedas de vecinos paralelas con centros aleatorios

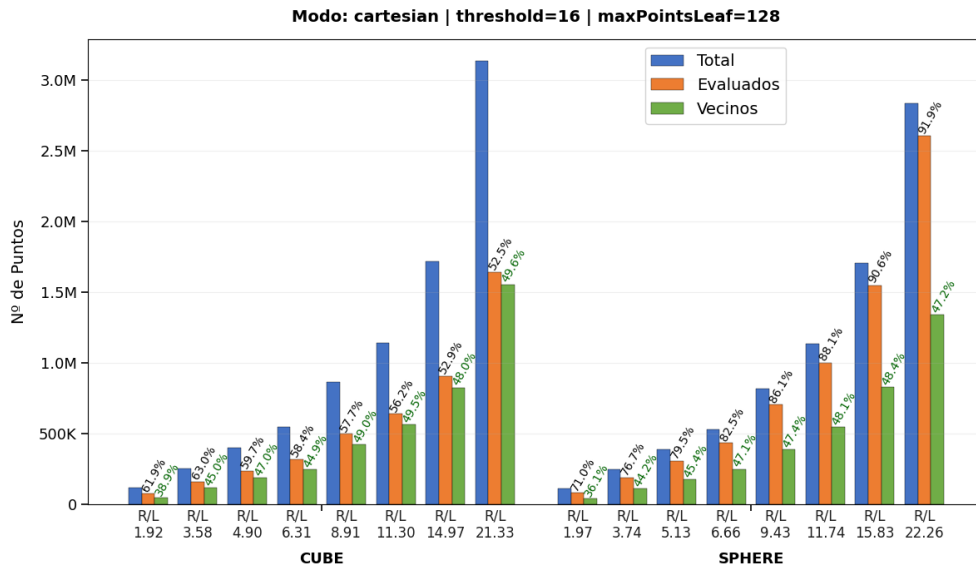
Estos experimentos exploran la eficiencia temporal en la ejecución de un lote de 10.000 consultas espaciales con centros distribuidos aleatoriamente. Cada configuración se repitió 5 veces, tomando como métrica el tiempo medio de ejecución. Cabe destacar que se empleó una paralelización a escala de 40 núcleos en todas las ejecuciones. Se usaron todas las nubes de puntos descritas en el apartado 3.4, variando los siguientes hiperparámetros:

- **Modo de reordenamiento.** Se probaron tanto los dos modos de poda propios como la versión de búsqueda sobre un Octree lineal base propuesta por Viñambres et al [?].
- **Algoritmo de búsqueda.** Se sometieron a pruebas los dos métodos estudiados y modificados: *neighborsPrune* y *neighborsStruct*.
- **Kernel.** Los dos kernels tridimensionales soportados: cubo y esfera.
- **Algoritmos de SFC.** Se usaron tanto los códigos de Morton como los de Hilbert. Sin embargo, no se analizaron los resultados por separado, eso está fuera del alcance del trabajo. Se incluyeron para promediar los resultados con los dos *encoders* y tener una visión más contrastada de los resultados.
- **Radio.** Se dividieron las nubes en tres grupos: de densidad alta, media y baja, según la naturaleza de la nube. Para cada grupo se ejecutó una selección de 8 radios distintos adaptada a la cantidad de vecinos aproximada que producirán.
- **Tamaño de hoja (número máximo de puntos).** Este parámetro es de los más críticos para este trabajo. Puede influir directamente en el rendimiento temporal de la poda. Se probaron valores de 128, 256, 512, 768, 1024
- **Umbral de poda (threshold).** Parámetro dedicado de esta optimización. Puede presentar cambios en el rendimiento, midiendo en qué punto deja de ser rentable o no el algoritmo de poda. Se probaron los valores 16, 64, 100.

5.2. RENDIMIENTO DE BÚSQUEDAS DE VECINOS PARALELAS CON CENTROS ALEATORIOS



(a) Modo de reordenamiento polar



(b) Modo de reordenamiento cartesiano

Figura 5.1: Efectividad teórica de cada modo de poda sobre el dataset *Lille_0*, sobre kernels cúbicos y esféricos de diferentes tamaños, usando un tamaño máximo de hoja de 128 puntos.

A continuación, se muestran los resultados obtenidos. Para empezar, se visualiza el tiempo de ejecución medio, dado un radio y una geometría de búsqueda, de cada combinación de algoritmo y reordenación local para los diferentes tamaños de hoja propuestos. En las Figuras ?? y ?? se ven los resultados variando el radio y la nube de puntos usada.

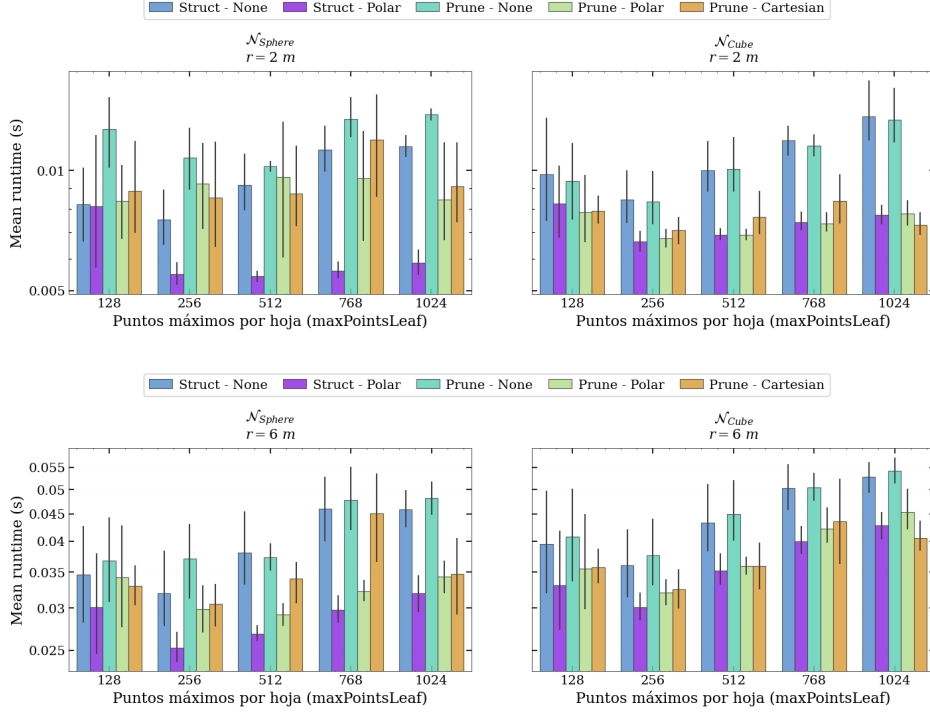


Figura 5.2: Tiempos medios de búsqueda de vecindades de 10.000 puntos aleatorios tomando diferentes radios en 5085_54320

Tal y como se evidencia en las Figuras ?? y ?? , los mínimos absolutos de tiempo se alcanzan de manera consistente al configurar un tamaño de nodo hoja fijado en 256. Asimismo, destaca la superioridad del algoritmo *neighborsStruct*. Aislando sus dos versiones, las implementaciones que incorporan reordenaciones locales y selectores de rango reportan una aceleración notable del tiempo de ejecución respecto al algoritmo original del Octree lineal, en especial en kernels esféricos.

En cuanto a los resultados aislados de *neighborsPrune*, en general están alejados, pero tanto el reordenamiento polar como el cartesiano tienen un desempeño similar, en ciertos casos mejor, al algoritmo base. Comparándolos entre ellos, ninguno de los dos reordenamientos aplicados a *neighborsPrune* destaca claramente sobre el otro.

Una tendencia que sí se puede apreciar es que los reordenamientos mantienen el rendimiento de forma más estable a medida que se disparan los tamaños de las hojas, contrastando con la degradación en los tiempos que sufren las implementaciones originales.

Seguidamente, se analiza el impacto combinado de las dos variables de control críticas en el rendimiento de la poda: el tamaño máximo de nodo hoja

5.2. RENDIMIENTO DE BÚSQUEDAS DE VECINOS PARALELAS CON CENTROS ALEATORIOS

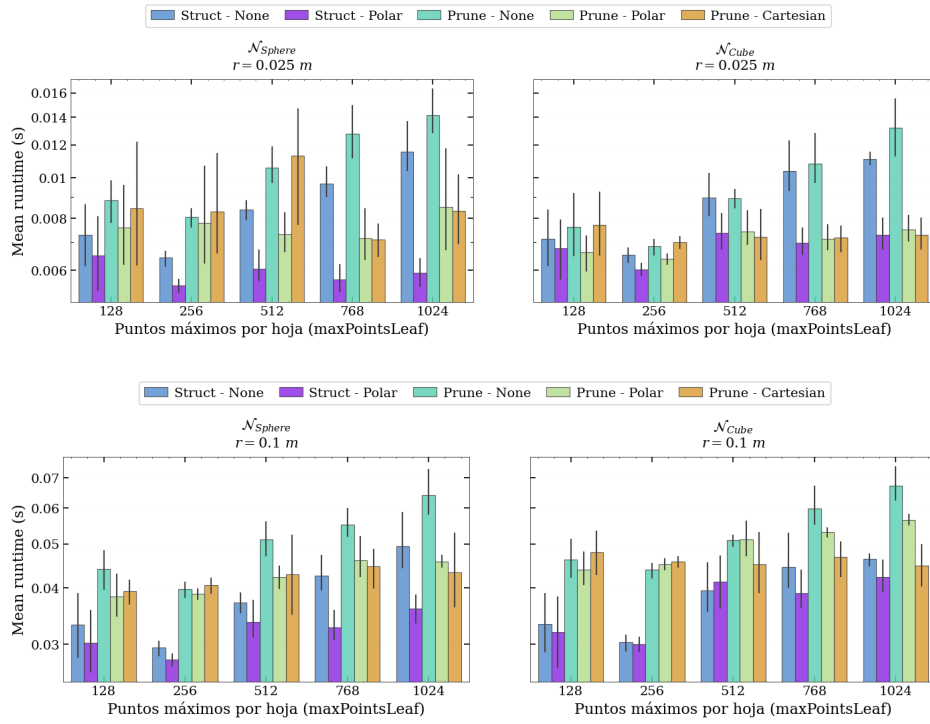


Figura 5.3: Tiempos medios de búsqueda de vecindades de 10.000 puntos aleatorios tomando diferentes radios en *bildstein_station1*

(**maxLeaf**) frente al umbral de activación del selector (**threshold**). Para aislar el efecto del resto de parámetros, ambas variables se contraponen en una matriz bidimensional.

El valor indexado en cada celda de la matriz corresponde a la media aritmética del *speedup* (S) obtenido para el conjunto de radios evaluados. Para evitar que los radios con mayores tiempos absolutos sesguen el mapa de calor, el *speedup* de cada radio r se normaliza dividiendo el tiempo de la configuración actual entre el tiempo mínimo absoluto registrado para ese mismo radio entre todo el espectro de hiperparámetros testados. Finalmente, este indicador se promedia combinando los resultados de ambos kernels de búsqueda (cubo y esfera), respondiendo a la siguiente expresión:

$$S_{\text{medio}} = \frac{1}{|K| \cdot |R|} \sum_{k \in K} \sum_{r \in R} \frac{T_{\text{optimo}}(k)}{T(k, r, \text{maxLeaf}, \text{threshold}, \text{algoritmo})} \quad (5.1)$$

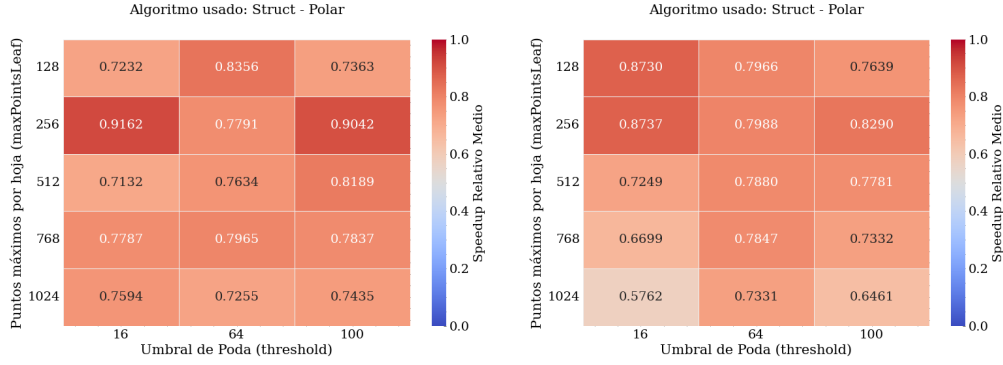


Figura 5.4: Análisis del rendimiento temporal de hiperparámetros en las nubes de puntos *Mar18Train* y *Mar18Test*

Debido a que la reordenación polar en conjunto con las búsquedas mediante *neighborsStruct* fue la más eficiente, será esa configuración en la que se centrará el estudio de hiperparámetros. Como se puede apreciar en la Figura ??, la combinación más eficiente para el comportamiento del Octree en las nubes del dataset *Hessigheim 3D* se produce con hojas de 256 puntos y con un umbral de activación de poda (**threshold**) fijado en 16 puntos.

No obstante, distintas distribuciones de puntos en cada dataset provocan que las combinaciones óptimas presenten ligeras variaciones según la nube de puntos analizada. Por este motivo, se evaluó de forma independiente qué par de hiperparámetros favorecía el rendimiento de cada entorno específico.

Estas configuraciones óptimas personalizadas constituyen las bases de datos con las que se alimentaron los resultados a continuación expuestos. En dichos gráficos se cuantifica el tiempo medio de ejecución de las 10.000 consultas espaciales a medida que se incrementa el radio de búsqueda de forma continua, distribuyendo cada serie como una combinación única de algoritmo de asignación y modo de reordenamiento local. Se pueden ver los resultados para diferentes nubes de puntos en las Figuras ?? y ??.

5.3. RENDIMIENTO DE BÚSQUEDAS DE VECINOS PARALELAS SOBRE TODA LA NUBE37

Estos valores temporales muestran una continuación de los resultados propuestos en las Figuras ?? y ??, pero pudiéndose apreciar tendencias con la introducción de la variación de radio y presentándose, además, comportamientos disjuntos entre nubes. En el dataset *Mar18_Train* los tiempos siguen una distribución notablemente uniforme. Por otro lado, los valores asociados al algoritmo *neighborsStruct* con poda local son los que dominan en cuanto a eficiencia temporal. Aún así, las podas con el algoritmo de *pruning* básico (destacando el modo cartesiano) presentan un rendimiento equiparable en radios intermedios. Eso sí, en ambos datasets estas ejecuciones se degradan en gran medida al escalar los tamaños de las figuras geométricas de búsqueda.

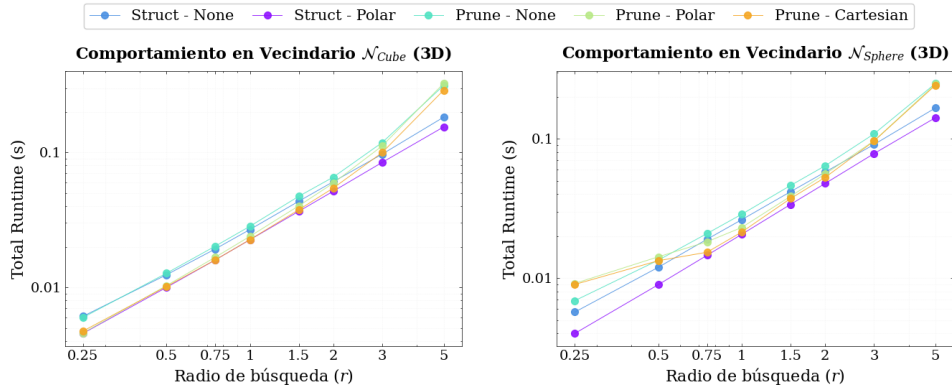


Figura 5.5: Tiempos medios de búsqueda de vecindades de 10.000 puntos aleatorios para diferentes radios en la nube *Mar18_Train*

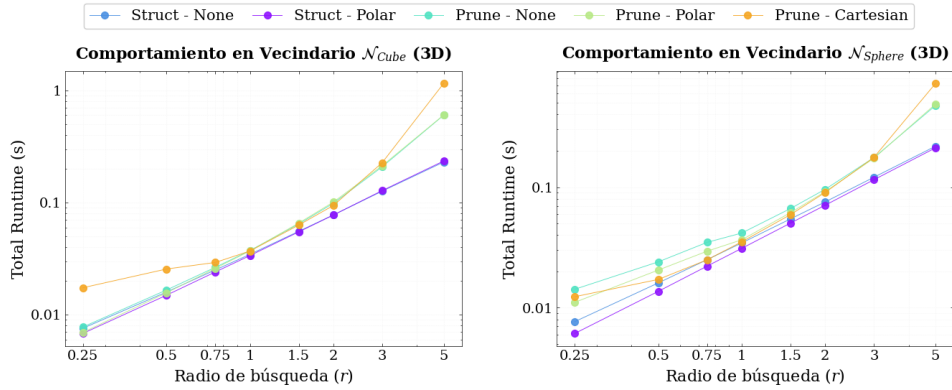


Figura 5.6: Tiempos medios de búsqueda de vecindades de 10.000 puntos aleatorios para diferentes radios en la nube *Mar18_Test*

5.3. Rendimiento de búsquedas de vecinos paralelas sobre toda la nube

A continuación, se trata un caso de uso muy común en el manejo de nubes de puntos 3D. Estos experimentos evalúan el comportamiento del sistema an-

te búsquedas masivas secuenciales sobre la totalidad de la nube. Debido a la naturaleza del recorrido y el reparto de carga contiguo por hilos, es necesario explorar las dinámicas de estas consultas de forma independiente. Ya que estas ejecuciones son muy costosas, se seleccionaron un subconjunto de las nubes preparadas, manteniendo variedad entre LiDAR terrestre y aéreo provenientes de diversos datasets. También se mantuvieron fijos los hiperparámetros de `maxLeaf` y `threshold`. Se escogió para cada conjunto de nubes un par de valores para estas variables de control que fuese exitoso en los experimentos de centros aleatorios:

- **Perfil de alta densidad:** Nube *bildstein_station1*. Configuración: [256, 64]
- **Perfil de densidad media:** Nubes *Mar18_train* y *Paris-Luxembourg_6*. Configuración: [128, 64]
- **Perfil de baja densidad:** Nubes *PNOA_2024_PNR_489-4672_NPC01* y *5085_54320*. Configuración: [256, 64]

Adicionalmente, se fijó el algoritmo SFC a los códigos de Hilbert y se fijó una única repetición de las búsquedas.

En la Figura ?? se exploran todos los resultados conseguidos.

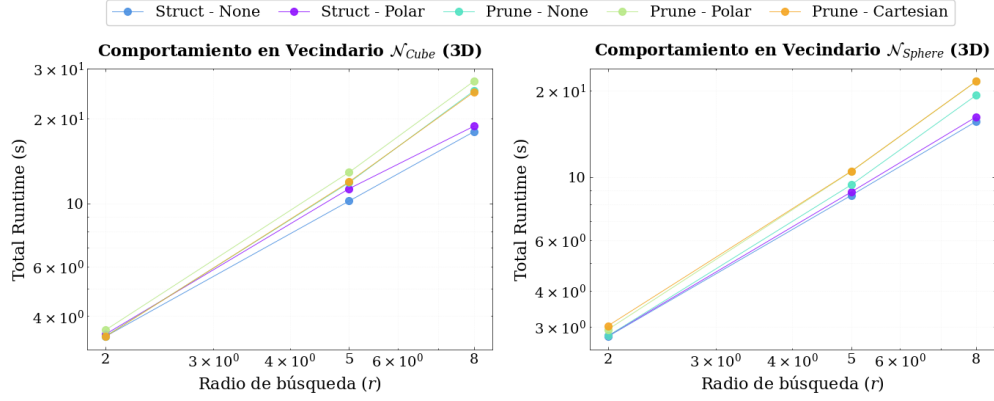
Estas Figuras muestran un patrón claro entre ellas, a la vez que difieren de los resultados de búsquedas en subconjuntos. El algoritmo *neighborsStruct* se mantiene como la mejor implementación. El rendimiento de los algoritmos con reordenamientos, para ambos kernels, es parejo pero ligeramente peor a lo largo de los datasets. Las dos formas de poda local existentes en *neighborsPrune* también funcionan de forma similar. Varían ligeramente entre kernels, destacando la forma cartesiana algo más en los kernels cúbicos, pero sin producir una tendencia muy clara. De forma general, se puede afirmar que las reordenaciones tienen menos impacto en búsquedas completas secuenciales que en consultas aleatorias.

5.4. Eficiencia del paralelismo sobre búsquedas de vecinos

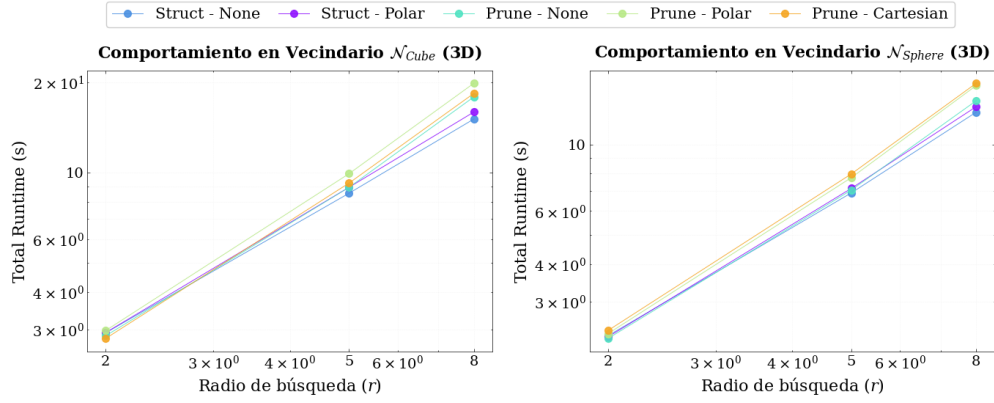
De forma complementaria, se estudió el rendimiento al modificar el número de hilos operando de forma paralela en las búsquedas. Para aislar lo máximo posible la escalabilidad de hilos se dejaron fijos los siguientes parámetros:

- Kernel - esférico
- Codificador - Hilbert
- Tamaño de hoja - 256
- Umbral de poda - 64
- Algoritmo - *neighborsPrune*

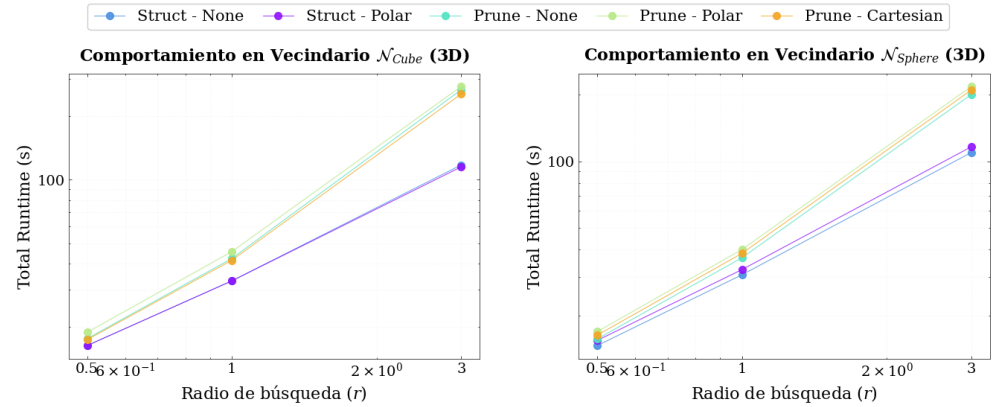
5.4. EFICIENCIA DEL PARALELISMO SOBRE BÚSQUEDAS DE VECINOS³⁹



(a) Dataset *5085_54320*



(b) Dataset *PNOA_2024_PNR_489-4672_NPC01*



(c) Dataset *Mar18_train*

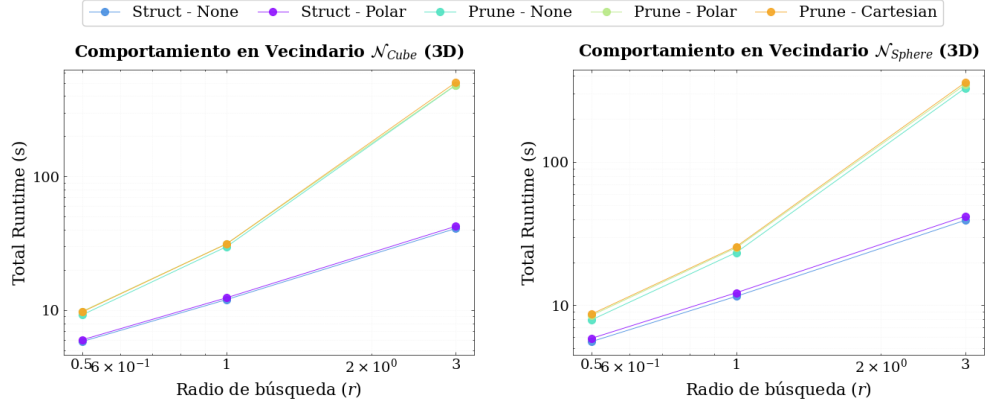
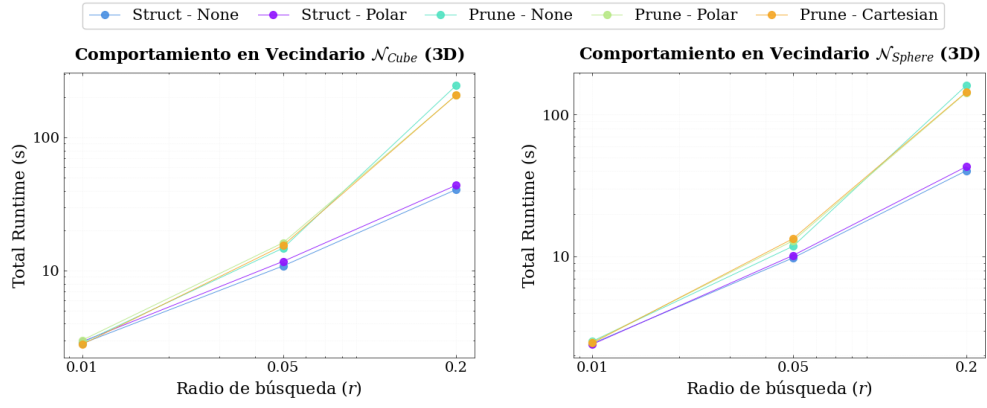
(d) Dataset *Paris_Luxembourg_6*(e) Dataset *bildstein_station1*

Figura 5.7: Tiempos de ejecución de búsquedas completas variando el radio operativo sobre kernels esféricos y cúbicos.

Cabe comentar la decisión de aislar el algoritmo. Se hizo así porque es el único que incluye los dos reordenamientos implementados y las diferencias de almacenamiento de vecinos no supone un factor crítico en materia de paralelización. Por otro lado, sí se introdujo un conjunto de tres tamaños de kernel en cada caso para estudiar el impacto a medida que crece el número de hojas visitadas y vecinos encontrados.

Las pruebas de escalabilidad, al ser ejecutadas en nodos de cómputo del CESGA, son realizadas sobre nodos de memoria NUMA (*Non-Uniform Memory Access*). En esta arquitectura, la latencia de acceso viene definida por la proximidad física entre el núcleo de la CPU que ejecuta el hilo de OpenMP y el banco de memoria donde residen los datos del Octree.

Para mitigar el sesgo introducido por la asignación aleatoria de páginas de memoria por parte del sistema operativo, todas las pruebas de paralelismo se confinaron utilizando la herramienta `numactl` [?].

Específicamente, se aplicó la directiva `--interleave=all`. Esta configuración fuerza un intercalado uniforme de las páginas de memoria virtual empleadas a lo largo de todos los nodos NUMA disponibles. Así, se distribuyen entre los dos sockets del nodo de computación las páginas correspondientes a la estructura del Octree, las réplicas de puntos reordenadas y el resto de variables con memoria reservada en tiempo de ejecución. Esto elimina el sesgo al variar el número de hilos, de forma que siempre se reparta la memoria de forma uniforme y equitativa, optimizando además el ancho de banda al no sobrecargar ninguno de los dos sockets por nodo en exceso.

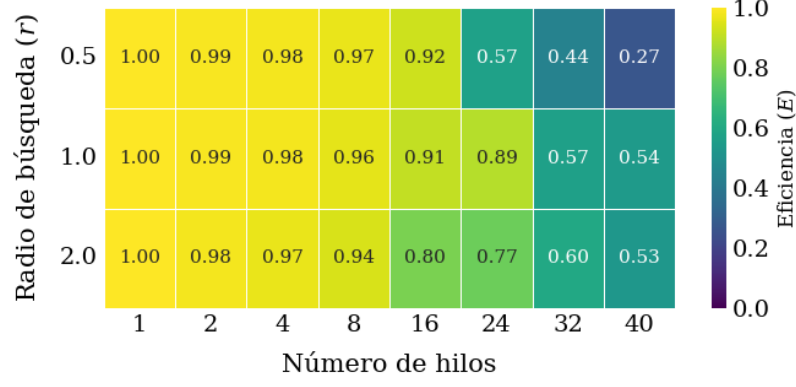
Los valores de eficiencia en la paralelización se calcularon a partir del tiempo de ejecución usando un solo núcleo de procesamiento, mediante la siguiente fórmula:

$$Eficiencia_{N_{proc}} = \frac{T_1}{T_{N_{proc}} * N_{proc}} \quad (5.2)$$

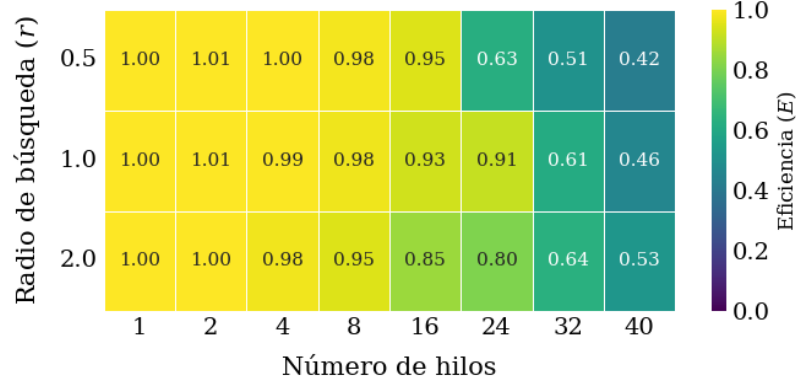
Al igual que el estudio del rendimiento temporal, se separarán los resultados entre búsquedas de subconjuntos aleatorias y búsquedas completas sobre la nube.

En los mapas de calor de la Figura ?? se muestra la variación de la escalabilidad para el dataset *Paris_Luxembourg_6* para 10.000 búsquedas de centros aleatorios. Se pueden observar valores similares en los 3 *heatmaps*, manteniéndose una eficiencia casi ideal para paralelización de hasta 8 hilos. A partir de 16 núcleos trabajando de forma paralela, existe una degradación, bajando la eficiencia del 90 % y llegando a tomar valores de entre 0.4 y 0.6 para paralelización de 32 y 40 CPUs. Para realizar una comparativa entre los reordenamientos, las diferencias de rendimiento se hacen más palpables cuantos más hilos se estén usando y cuanto más rápida sea la ejecución (radios más cortos). En estos casos el porcentaje del tiempo gastado en el manejo del paralelismo es naturalmente mayor al ser más breves las operaciones en sí. En esa sección superior derecha de las matrices, la búsqueda base presenta un peor comportamiento que los algoritmos con poda implementada.

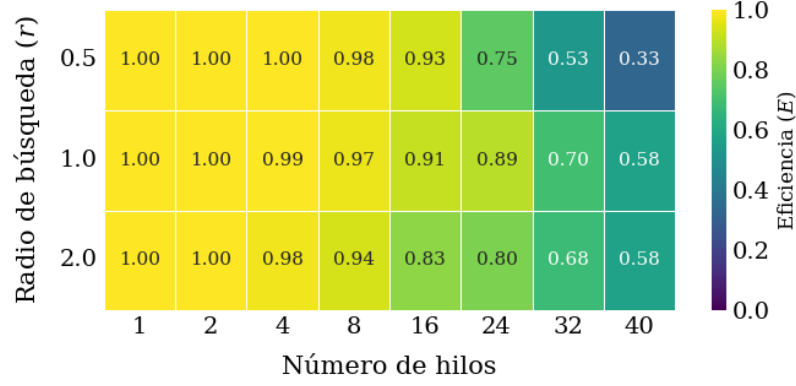
Para terminar, se exponen en la Figura ?? los mapas de calor para el dataset *Lille_0*, sobre el que se realizaron búsquedas completas para los distintos reordenamientos. En estos *heatmaps* se aprecia una paralelización muy potente,



(a) Algoritmo de búsqueda de Octree lineal base



(b) Reordenamiento de las hojas del Octree en modo polar

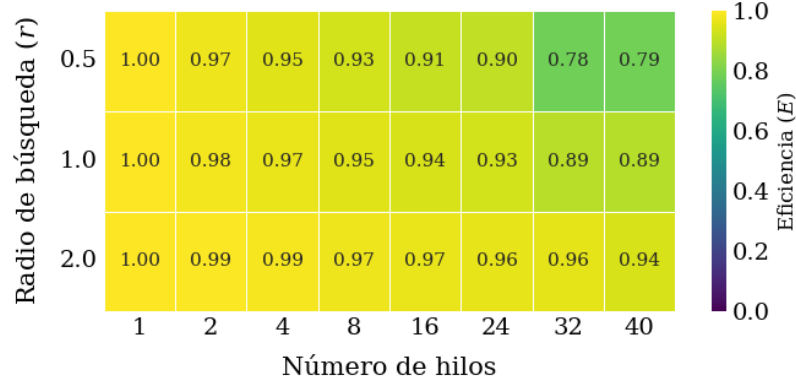


(c) Reordenamiento de las hojas del Octree en modo cartesiano

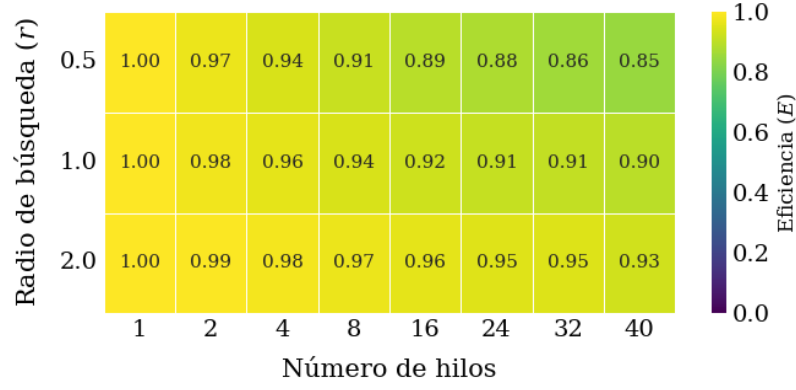
Figura 5.8: Mapas de calor de eficiencia de paralelización sobre centros aleatorios en la nube de puntos *Paris_Luxembourg* _6

5.4. EFICIENCIA DEL PARALELISMO SOBRE BÚSQUEDAS DE VECINOS⁴³

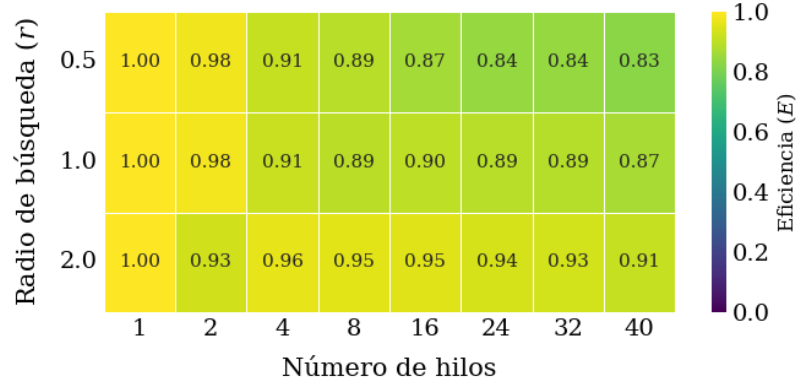
solamente bajando del 90 % de eficiencia en los casos más extremos, con más de 32 núcleos de procesamiento paralelos y radios cortos. De nuevo, se debe poner el foco en la zona superior derecha para observar los resultados más correlacionados con el funcionamiento general de la escalabilidad. Al igual que en las búsquedas aleatorias, las ejecuciones sin poda a nivel de hoja tienen peor eficiencia temporal en los casos más críticos de paralelización, sobre un 5% más bajo que en los modos del reordenamiento polar y cartesiano, que tienen comportamientos casi idénticos entre sí.



(a) Algoritmo de búsqueda de Octree lineal base



(b) Reordenamiento de las hojas del Octree en modo polar



(c) Reordenamiento de las hojas del Octree en modo cartesiano

Figura 5.9: Mapas de calor de eficiencia de paralelización sobre coberturas completas en la nube de puntos *Lille_0*

Capítulo 6

Discusión de los Resultados

En este capítulo se someten a un análisis crítico los resultados empíricos expuestos en el capítulo previo. El propósito fundamental es evaluar la eficiencia, viabilidad y éxito del diseño e implementación de las técnicas de poda a nivel de nodo hoja sobre la arquitectura del Octree lineal en relación a los resultados y soluciones existentes a día de hoy para la búsqueda de vecindades en nubes de puntos capturadas con tecnología LiDAR.

6.1. Análisis de Estructuras de Almacenamiento: algoritmo *neighborsStruct* frente a *neighborsPrune*

Como tendencia independiente de las contribuciones directas de este trabajo, se constata que la arquitectura de almacenamiento de puntos vecinos basado en pares de índices, (*neighborsStruct*), reporta de manera sistemática una eficiencia temporal superior a las inserciones secuenciales (presentes en *neighborsPrune*). Este comportamiento se explica por la reducción de inserciones en el vector *ptsInside* realizadas, diferencia que se acentúa más cuantos más puntos consecutivos sean considerados válidos.

Sin embargo, el hallazgo más relevante de esta comparativa radica en el comportamiento asimétrico que manifiesta el algoritmo de poda polar. Esta optimización exhibe un impacto sustancialmente mayor al acoplarse sobre *neighborsStruct* que al operar sobre *neighborsPrune*. La hipótesis que lo justifica reside en el modo de inserción en el vector de resultados:

- En la variante básica *neighborsPrune*, el algoritmo evalúa secuencialmente los puntos y añade un elemento al contenedor de salida de forma individualizada si, y solo si, se verifica la condición de vecindad positiva. Bajo este esquema, la poda angular actúa únicamente reduciendo el coste de la evaluación *booleana* interna (*k.isInside()*), dado que todos los puntos descartados habrían devuelto un valor falso en cualquier caso. El ahorro computacional queda limitado a la supresión de la comprobación aritmética de *isInside()*.
- Por el contrario, en *neighborsStruct* la lógica opera por bloques de rangos contiguos mediante la guarda condicional `if (!k.isInside(points[i]))`.

Si un punto no pertenece al entorno, se introduce en el contenedor el rango completo indexado previamente. Al efectuar la poda a nivel de hoja, cuanto más efectiva sea la discriminación geométrica, menor será el número de ocasiones en las que el flujo de ejecución entra en esta bifurcación, reduciendo masivamente las operaciones de inserción y de redimensionado del vector de salida.

Esta divergencia es particularmente visible bajo kernels esféricos, ya que es la forma geométrica adaptada al cálculo de poda angular que se realiza en el modo polar. Aparte, en nubes de puntos con una cierta densidad espacial (tales como *Lille_0*, *bildstein_station1*, *sg27_station8* o *Mar18_test*), se aprecia que a medida que se incrementa el radio operativo, el número de puntos consecutivos que cumplen con el criterio de inclusión se dispara. En estas regiones de alta saturación, el algoritmo de pares de índices de *neighborsStruct* amplía la diferencia proporcionalmente en rendimiento contra el *pruning* estándar.

6.2. Sensibilidad al Tamaño de Hoja y Dinámicas del Kernel

Un hito de la arquitectura propuesta es su capacidad para romper la degradación del rendimiento asociada al crecimiento del tamaño de los nodos hoja (*maxLeaf*). En las implementaciones secuenciales clásicas, incrementar la capacidad del nodo hoja implica obligatoriamente dilatar el tiempo de cómputo, al verse forzado el sistema a realizar un escaneo lineal sobre un mayor volumen de puntos desordenados.

Con la introducción de los modos de reordenamiento local, el coste temporal deja de depender directamente del número absoluto de puntos contenidos en la hoja. El rendimiento pasa a depender de la distribución espacial y del porcentaje de descarte geométrico efectivo. Este porcentaje de descarte fue estudiado con detalle en el capítulo anterior, y se pueden extraer una serie de aprendizajes sobre el funcionamiento teórico, más allá de los tiempos de ejecución producidos. El reordenamiento local polar funciona mejor en combinación con el modo de kernel esférico por la concordancia geométrica existente, mientras que para el caso de los cubos hay que trazar una esfera que lo circunscriba para poder hallar esos límites. Además, también se entiende la mayor efectividad de poda en radios bajos. La sección angular mínima que contiene a la esfera de búsqueda es más cerrada cuánto más pequeña sea esa esfera. Esta tendencia esperada sí se corresponde con los resultados del algoritmo *neighborsStruct* en las Figuras ?? y ?. De forma complementaria, la efectividad de la poda en los experimentos estudiados para el reordenamiento cartesiano tuvo éxito en las búsquedas de geometría cúbica, al separar perfectamente los puntos según los ejes ordenados. En estos casos, se hallaba una dinámica contrapuesta al variar el tamaño del radio: las podas son más débiles en radios pequeños. Esto tiene una explicación geométrica detrás, y es que en búsquedas cúbicas, si el kernel solo corta la hoja en una dimensión la separación de puntos entre vecinos y no vecinos será perfecta. Solo en los casos en los que dos o tres dimensiones cortan

el nodo se producirán podas imperfectas. En cubos grandes (proporcionalmente a los tamaños de los nodos terminales) más octantes tendrán la cualidad de ser cortados solamente por una dimensión, con lo que más puntos serán separados de forma perfecta. Sin embargo, este comportamiento no se aprecia directamente en los experimentos realizados usando el algoritmo *neighborsPrune*, y tiene una explicación que se introducirá a continuación.

6.3. Evaluación de los Modos de Reordenamiento Local

Al contrastar de manera aislada los dos modos de reordenamiento implementados (polar y cartesiano) frente al algoritmo de búsqueda con inserciones secuenciales, las métricas reflejan diferencias muy sutiles. Esto se debe a que existe un beneficio extra en que se inserten una serie de puntos consecutivos, como sí pasa al usar *neighborsStruct*. A pesar de ello, la introducción de la poda logra amortizar con éxito su propia sobrecarga algorítmica, manteniendo registros competitivos.

Teóricamente, cabría esperar una divergencia de rendimiento más acusada entre ambas reordenaciones al cruzar sus kernels respectivos, dado que cada una minimiza la sobreestimación del volumen envolvente en su propio dominio geométrico. Sin embargo, la mejora temporal respecto a la versión secuencial es tan pequeña que el ahorro funcional apenas sobresale respecto al coste fijo de computar los límites de la hoja. Hay casos concretos en los que, tanto el modo polar como el cartesiano, ganan rendimiento al ser acoplados sobre *neighborsPrune*, pero no de forma consistente ni con una tendencia clara.

En este contexto en el que la sobrecarga prácticamente alcanza al ahorro conseguido mediante la poda, los escenarios cruzados (modo cartesiano sobre kernels esféricos y modo polar sobre kernels cúbicos) operan como un estado intermedio que tampoco degrada apenas el rendimiento. En estos casos, la sobreestimación geométrica del volumen impide que la mayoría de las hojas califiquen para un descarte. No obstante, estas hojas no aptas evitan la ejecución de la búsqueda binaria y se desvían de forma inmediata hacia el recorrido secuencial ordinario. El software no penaliza de forma crítica estas configuraciones subóptimas de entrada, garantizando la estabilidad del rendimiento.

6.4. Calibración del Espacio de Hiperparámetros

Los ensayos demuestran que el rendimiento de las consultas espaciales es altamente sensible al ecosistema de parámetros de control, condicionado de manera simultánea por la densidad de la nube, la magnitud del radio y la posición espacial del centro de consulta.

A pesar de esta volatilidad, es posible extraer una regla de diseño generalizada: la configuración de tamaños de nodo hoja superiores a `maxLeaf = 256` introduce un principio de degradación en la eficiencia temporal. Esta pérdida de rendimiento, cabe destacar, es significativamente atenuada en comparación con la bajada de rendimiento en los algoritmos clásicos. Aún así, delimita el umbral

que no se debería superar a menos que se esté trabajando sobre un entorno ya conocido y estudiado.

En lo que respecta al umbral mínimo de puntos para la activación de la poda (**threshold**), el valor de 64 puntos se consolida como el punto de equilibrio óptimo. Límites inferiores introducen un *overhead* ineficiente en la CPU al forzar el cálculo de selectores de rango para subconjuntos de puntos minúsculos cuya evaluación secuencial directa en memoria sería más veloz. Por el contrario, límites excesivamente elevados clausuran ventanas de oportunidad de la poda en hojas medianas que albergan buen potencial de descarte.

6.5. Dinámicas en Búsquedas de Cobertura Completa

El escenario de evaluación sobre búsquedas completas y secuenciales (*Full Searches*) proporcionó un escenario de resultados muy dispar a los discutidos hasta ahora. En este modo de exploración, la tasa de fallos de caché se reduce al mínimo debido a la asignación por bloques de puntos a cada hilo OpenMP.

Al procesarse núcleos de consulta vecinos de forma consecutiva, hay una gran tasa de reutilización de puntos en los niveles superiores de la memoria caché de la CPU tras las iteraciones previas. Este fenómeno se amplifica teniendo en cuenta que los puntos fueron reordenados de forma global mediante las curvas de Hilbert para mejorar su localidad. Como consecuencia directa de esta localidad hardware, el coste temporal de evaluar secuencialmente un conjunto de puntos que potencialmente se iban a descartar es extremadamente bajo.

Por ende, dado que el coste de cálculo del *overhead* para la selección de rangos geométricos se mantiene constante, el balance de rentabilidad de la poda es negativo. Los tiempos de ejecución totales, aunque no suponen una degradación notable, se sitúan ligeramente por detrás de las búsquedas con los algoritmos originales. En este contexto de coberturas completas, no se considera útil la implementación actual de reordenamientos locales, ya que no produce un rendimiento consistentemente mejor al código de partida a la vez que otorga a los *Octrees* un recorrido de búsqueda más complejo y produce un mayor gasto en memoria.

Capítulo 7

Conclusiones y posibles ampliaciones

7.1. Principales aportaciones y limitaciones

Tras el análisis crítico de los experimentos ejecutados, se presenta a continuación una síntesis de las principales conclusiones y aportaciones técnicas derivadas del diseño e implementación del sistema:

- **Sinergia de los reordenamientos con las estructuras de almacenamiento:** La poda local actúa de forma dependiente con la estrategia de gestión de memoria. Manifestó una ganancia crítica al acoplarse al algoritmo orientado a bloques (*neighborsStruct*), ya que la discriminación geométrica se especializa en agrupar puntos vecinos y reduce las bifurcaciones condicionales y las operaciones de inserción. Esto provoca que el reordenamiento cartesiano no produce resultados favorables al no poder acoplarse naturalmente a este sistema de inserciones en bloque, al contrario que el reordenamiento propuesto mediante el ángulo φ .
- **Especialización y dinámica dimensional de los kernels:** Se comprueba que la efectividad teórica del descarte está ligada a la morfología de la consulta. El modo polar maximiza su rendimiento en kernels esféricos (especialmente a radios bajos debido al estrechamiento de la sección angular), mientras que el modo cartesiano destaca en kernels cúbicos (particularmente a radios mayores, donde los cortes unidimensionales con las caras del nodo permiten una separación perfecta entre puntos vecinos y no vecinos).
- **Delimitación del espacio de hiperparámetros:** A pesar de la variabilidad y la naturaleza única de cada nube LiDAR, se consolida un umbral operativo óptimo fijando el **threshold** en 64 puntos, cota que evita introducir un *overhead* ineficiente en la CPU ante subconjuntos minúsculos. Asimismo, se establece que valores superiores a **maxLeaf** = 256 marcan el inicio de la degradación de la eficiencia, partiendo el espacio en octantes no lo suficientemente concretos. Aún así, esta implementación supone una

ganancia en flexibilidad a la hora de diseñar el tamaño de los octantes del *Octree*.

- **Inversión de rentabilidad en coberturas completas:** Se identifica una limitación física del algoritmo en las búsquedas completas. La ordenación global por curvas de Hilbert combinada con la asignación por bloques de OpenMP maximiza de forma nativa la reutilización de datos en la memoria caché de la CPU y el tiempo ahorrado con los descartes de puntos no llega a equilibrar la sobrecarga introducida.

A partir de estas premisas, se puede confirmar la mejora real introducida en este trabajo mediante la propuesta del reordenamiento basado en coordenadas esféricas integrado sobre el algoritmo *neighborsStruct*. Para poner en el contexto de la literatura técnica esta implementación, y tal como se abordó en el estado del arte, se tomaron como referencia de rendimiento los resultados del Octree lineal publicados por Viñambres et al. [?]. Dicha estructura base minimizaba los tiempos de ejecución en relación con las principales implementaciones abiertas de Octrees, tales como PCL, PicoTree, NanoFLANN y unibnOctree.

Por consiguiente, la integración original de la técnica de poda a nivel de hoja desarrollada en este proyecto sobre la arquitectura de Viñambres et al. supone, hasta donde se conoce y bajo el entorno operativo adecuado (consultas individuales/aleatorias con kernels esféricos), la solución más eficiente en la actualidad para detectar y almacenar vecinos sobre nubes de puntos tridimensionales LiDAR.

7.2. Vías de Mejora y Trabajo Futuro

Por último, como una ampliación a futuro, se propone de desarrollo el diseño de un mecanismo que permita acoplar la búsqueda de rangos cartesiana directamente sobre la infraestructura orientada a datos contiguos de *neighborsStruct*.

Esta integración tiene como objetivo desbloquear el gran potencial de la poda por ejes cartesianos sobre geometría cúbicas, visible en los porcentajes de puntos podados conseguidos, cuyo impacto no pudo transformarse en eficiencia temporal real. El motivo es la incompatibilidad en cuanto a estructura del algoritmo *neighborsStruct* base con los múltiples vectores reordenados usados simultáneamente durante las consultas. En caso de encontrar una solución para esta integración, se elevaría drásticamente la rapidez temporal de las consultas sobre kernels cúbicos.

Teóricamente, esta eficiencia sería incluso de mayor magnitud que la encontrada para kernels esféricos mediante la poda angular. Esto es debido a que la efectividad geométrica del descarte cartesiano es sustancialmente más alta: tal y como se constató en las métricas de depuración, apenas existen puntos evaluados por el algoritmo que no terminen cumpliendo el criterio de vecindad. Esto implica que el sistema haría todavía menos operaciones de cómputo de vecinos por hoja. Esto constituye, sin duda, la línea de investigación más prometedora para la continuidad de este trabajo.

Apéndice A

Manuais técnicos

En función do tipo de Traballo e metodoloxía empregada, o contido poderase dividir en varios documentos. En todo caso, neles incluírase toda a información precisa para aquelas persoas que desexen repetir o experimento (por exemplo código fonte, recursos necesarios, operacións necesarias para modificacións e probas, posibles problemas, etc.). O código fonte poderase entregar en soporte informático en formatos PDF ou postscript.

Apéndice B

Manual de usuario

Nota previa

Este manual documenta el uso del programa `octrees-benchmark-coordenadas-polares`, que parte de la implementación base del Octree lineal desarrollada por Viñambres et al. [?]. El presente trabajo se centra en el diseño e implementación de las técnicas de poda a nivel de nodo hoja sobre las estructuras de datos existentes (*Octrees*) y sus correspondientes algoritmos de búsqueda.

B.1. Instalación

B.1.1. Dependencias

LASTools En primer lugar es necesario instalar las dependencias de LASTools, listadas en <https://github.com/LAStools/LAStools>:

```
sudo apt-get install libjpeg62 libpng-dev libtiff-dev libjpeg-dev \
    libz-dev libproj-dev liblzma-dev libjbig-dev libzstd-dev \
    libgeotiff-dev libwebp-dev liblzma-dev libsqlite3-dev
```

A continuación, se clona el repositorio y se compila:

```
git clone --depth 1 https://github.com/LAStools/LAStools lib/LAStools
(cd lib/LAStools && cmake -B build -DCMAKE_BUILD_TYPE=Release \
    -DCMAKE_INSTALL_PREFIX=$PWD/../LASlib . && \
    cmake --build build -- -j && cmake --install build)
rm -rf lib/LAStools
```

PCL versión 1.15 (Opcional) Se debe obtener el código fuente de la versión 1.15 desde <https://github.com/PointCloudLibrary/pcl/releases> y compilarlo. La carpeta de instalación usada por defecto es `~/local/pcl`, aunque puede modificarse, junto con la versión buscada, editando el fichero `CMakeLibraries.cmake`.

```
wget https://github.com/PointCloudLibrary/pcl/releases/download/\
pcl-1.15.0/source.tar.gz
```

```
tar xvf source.tar.gz
cd pcl && mkdir build && cd build
cmake .. -DCMAKE_INSTALL_PREFIX=$HOME/local/pcl -DCMAKE_BUILD_TYPE=Release
make -j2
make -j2 install
```

Si PCL no se encuentra durante la compilación, el proyecto se compilará igualmente con normalidad, pero sin soporte para los algoritmos de *benchmark* basados en el Octree y el KD-Tree de PCL.

PAPI (Opcional, para perfilado de caché)

```
wget https://github.com/icl-utk-edu/papi/releases/download/\
papi-7-2-0-t/papi-7.2.0.tar.gz -P lib
tar xvf lib/papi-7.2.0.tar.gz -C lib && rm lib/papi-7.2.0.tar.gz \
&& mv lib/papi-7.2.0 lib/papi/
(cd lib/papi/src && ./configure --prefix=$(pwd)/.. && make -j \
&& make install)
```

B.1.2. Compilación

Una vez instaladas las dependencias necesarias, dentro del directorio del proyecto basta con ejecutar:

```
cmake -B build -DCMAKE_BUILD_TYPE=Release .
cmake --build build
```

Esto genera el ejecutable en `build/octrees-benchmark`.

Para más comodidad, existe el script de bash `compile.sh`, que ejecuta esos comandos además de limpiar el directorio de build (recomendado borrar el directorio siempre que se compile después de introducir cambios en el código).

B.1.3. Verificación de la instalación: Tests

El proyecto incluye una batería de tests unitarios mediante GoogleTest, que permite comprobar que la instalación y compilación se realizaron correctamente. Para habilitarla, se debe compilar el proyecto con la opción `-DBUILD_TESTS=ON`:

```
cmake -B build -DCMAKE_BUILD_TYPE=Release -DBUILD_TESTS=ON .
cmake --build build
```

Tras compilar el proyecto, todos los tests se pueden ejecutar mediante:

```
make # compila la biblioteca y todos los ejecutables de test
ctest --output-on-failure
```

También es posible ejecutar cada batería de tests de forma individual, por ejemplo:

```
./tests/test_points
./tests/test_encoders
./tests/test_octree
./tests/test_octree_advanced
```

Para ejecutar un caso de test concreto con CTest, se utiliza la opción `-R` seguida del nombre del test, por ejemplo:

```
ctest -R LinearOctreeTest.RadiusSearch
```

Adicionalmente, bajo `tests/example.cpp` se incluye un pequeño ejemplo de uso de las funcionalidades de la biblioteca, que se compila automáticamente junto con el proyecto principal:

```
make test_library
./tests/test_library
```

B.2. Ejecución

El programa se ejecuta a través del binario generado en la compilación, `build/octrees-benchmark`, configurando su comportamiento mediante parámetros de línea de comandos. A continuación se muestra un ejemplo de invocación, correspondiente a uno de los experimentos descritos en esta memoria: una búsqueda completa secuencial de vecinos sobre la nube `Paris Luxembourg_6`, comparando los distintos reordenamientos existentes sobre los dos algoritmos de búsqueda sobre los que se trabajó para una selección de radios.

```
./build/octrees-benchmark --kernels "sphere" -i "data/paris_lille/Lille_0.las" \
  -o "out_tfg_reorders_v3/full" -e "hilb" --kernels "cube,sphere" -r "2.0,5.0,8.0" -
```

Para reproducir el conjunto completo de experimentos descritos en los capítulos de Pruebas y Discusión de Resultados de esta memoria, el proyecto incluye una serie de scripts:

- `bench_neighbors_reorders.bash`: búsquedas de vecinos de centros aleatorios.
- `bench_neighbors_full_reorders.bash`: búsquedas de vecinos de coberturas completas.
- `bench_neighbors_parallel.bash`: búsquedas de vecinos para diferentes escalas de paralelización, tanto de centros aleatorios como coberturas completas.
- `bench_neighbors_debug_range_selector.bash`: búsquedas reducidas (60 búsquedas), sobre el mismo conjunto de nubes, con la opción de depuración de podas habilitadas, devolviendo archivos con el número de puntos podados por hoja. (Modificar con precaución: generan archivos muy pesados, escribiendo una línea por hoja evaluada en cada búsqueda).

- Adicionalmente, se encuentran los scripts del estudio de partida:
- `bench_locality.bash`,
- `bench_memory.bash`,
- `bench_neighbors.bash`,
- `bench_neighbors.temp.bash`

Asimismo, bajo la carpeta `plots_reorders` se incluyen todos los scripts y notebooks en Python empleados para generar las figuras presentadas en esta memoria. Se puede consultar cada gráfica para cada una de las nubes de puntos estudiadas, ya que no todas fueron incluidas en la memoria. En la carpeta `plots` se encuentran las figuras de los resultados publicados por Viñambres et al., también disponibles para consulta.

B.2.1. Formato de entrada

El programa toma como entrada nubes de puntos en formato binario `.LAS`, gestionadas internamente mediante la biblioteca `LASTools`.

B.2.2. Formato de salida

Los resultados de cada ejecución se almacenan en la ruta indicada mediante el parámetro `-o/--output`. El programa genera ficheros con las métricas temporales y de efectividad de poda correspondientes a la configuración de parámetros especificada, que sirven de entrada a los scripts de generación de gráficas.

B.2.3. Ejecución de la verificación de funcionamiento de las podas

En la ruta `test_funcionalidad_podas/test_vecindades.ipynb` está el test de Python que comprueba si los resultados usando reordenamientos son idénticos a los resultados sin hacer uso de la optimización (función `verificar_integridad_vecinos`). Los parámetros a introducir son los siguientes:

- **data_path**: Ruta de los archivos `.csv` devueltos por las ejecuciones del programa.
- **clouds**: El test comprobará todos los casos posibles solamente para las nubes aquí especificadas.

B.3. Parámetros del programa

A continuación se detallan todos los parámetros aceptados por el programa. Para consultar esta misma información desde la propia terminal, se puede invocar la ayuda integrada mediante:

```
./build/octrees-benchmark --help
```

Cuadro B.1: Parámetros principales del programa `octrees-benchmark`.

Opción	Alias	Descripción
-h	--help	Muestra el mensaje de ayuda.
-i	--input	Ruta del fichero de entrada.
-c	--container-type	Tipo de contenedor a utilizar. Por defecto: AoS. Valores posibles: SoA, AoS.
-o	--output	Ruta del fichero de salida.
-r	--radii	Radios de búsqueda a evaluar (separados por comas, p. ej. 2.5,5.0,7.5).
-v	--kvalues	Valores de k para las búsquedas KNN (separados por comas, p. ej. 10,50,250,1000).
-s	--searches	Número de búsquedas a realizar (con centros aleatorios, salvo que se indique --sequential). Si se indica all, se realiza una búsqueda sobre la totalidad de la nube (con indexación secuencial).
-t	--repeats	Número de repeticiones a realizar para cada configuración del <i>benchmark</i> .
-k	--kernels	Especifica los kernels de búsqueda a utilizar (separados por comas o all). Valores posibles: sphere, cube, square, circle.

Opción	Alias	Descripción
-a	--search-algo	Especifica los algoritmos de búsqueda a ejecutar (separados por comas o <code>all</code>). Por defecto: <code>neighborsPtr,neighbors,neighborsPrune,neighborsStruct</code>. Valores posibles: Búsqueda por radio: • <code>neighborsPtr</code> – búsqueda básica sobre el Octree de punteros • <code>neighbors</code> – búsqueda básica sobre el Octree lineal • <code>neighborsPrune</code> – búsqueda optimizada sobre el Octree lineal con poda de octantes • <code>neighborsStruct</code> – búsqueda optimizada mediante rangos de índices • <code>neighborsApprox</code> – búsqueda aproximada (cotas superior/inferior), requiere <code>--approx-tol</code> • <code>neighborsUnibn</code> – búsqueda mediante <code>unibnOctree</code> • <code>neighborsPCLKD</code> – búsqueda mediante el KD-Tree de PCL (si está disponible) • <code>neighborsPCLOct</code> – búsqueda mediante el Octree de PCL (si está disponible) • <code>neighborsPico</code> – búsqueda mediante PicoTree Búsqueda KNN: • <code>KNNV2</code> – búsquedas KNN sobre el Octree lineal • <code>KNNNanoflann</code> – búsquedas KNN mediante <code>nanoflann</code> • <code>KNNPCLKD</code> – búsqueda KNN mediante el KD-Tree de PCL (si está disponible) • <code>KNNPCLOct</code> – búsqueda KNN mediante el Octree de PCL (si está disponible) • <code>KNNPico</code> – búsqueda KNN mediante PicoTree
-e	--encodings	Selecciona las codificaciones SFC con las que reordenar la nube antes de las búsquedas (separadas por comas o <code>all</code>). Por defecto: <code>all</code>. Valores posibles: • <code>none</code> – sin codificación; el Octree lineal no se construirá con esta opción • <code>mort</code> – reordenamiento mediante SFC de Morton • <code>hilb</code> – reordenamiento mediante SFC de Hilbert

Opción	Alias	Descripción
--	--local-reorders	Selecciona los reordenamientos locales a nivel de hoja a aplicar sobre la nube antes de las búsquedas (separados por comas o <code>all</code>). Por defecto: <code>all</code> . Valores posibles: • <code>none</code> – sin reordenamiento local • <code>polar</code> – reordenamiento local polar (según el ángulo azimutal φ) • <code>cartesian</code> – reordenamiento local cartesiano (según las coordenadas x, y, z)
-u	--umbral-poda	Umbral mínimo de puntos (<code>threshold</code>) necesario para emplear el selector de rango y podar la hoja. Por defecto: 16. Toda hoja con un número de puntos inferior a este umbral se recorrerá de forma secuencial, sin aplicar la poda.
--	--debug-ranges	Activa el registro (<i>logging</i>) detallado de la selección de rangos en cada hoja conflictiva visitada durante las búsquedas.
--	--debug	Activa el modo de depuración general (mide los tiempos de construcción del Octree y de codificación).
--	--build-enc	Ejecuta <i>benchmarks</i> para la codificación y construcción de las estructuras seleccionadas (aquellas con representante en <code>-a / --search-algo</code>).
--	--memory	Ejecuta un <i>benchmark</i> simple para medir la memoria consumida por una estructura, mediante perfilado del <i>heap</i> . Valores posibles: <code>ptrOct</code> , <code>linOct</code> , <code>unibnOct</code> , <code>nanoKD</code> , <code>pclOct</code> , <code>pclKD</code> , <code>picoTree</code> .
--	--locality	Ejecuta <i>benchmarks</i> para analizar la localidad de la nube de puntos tras aplicar los reordenamientos indicados.
--	--cache-profiling	Activa el perfilado de la memoria caché durante la ejecución de los algoritmos de búsqueda, mediante PAPI.
--	--check	Activa la comprobación de resultados (opción heredada; se recomienda usar <code>avg_result_size</code> para verificar la corrección de los resultados).
--	--no-warmup	Desactiva la fase de calentamiento (<i>warmup</i>) previa a las mediciones.
--	--approx-tol	Valores de tolerancia para la búsqueda aproximada (separados por comas, p. ej. <code>10.0,50.0,100.0</code>).

Opción	Alias	Descripción
--	--num-threads	Lista de números de hilos a evaluar en el test de escalabilidad (separados por comas, p. ej. 1,2,4,8,16,32). Si no se especifica, OpenMP emplea por defecto el número máximo de hilos disponibles y no se ejecuta ningún test de escalabilidad.
--	--sequential	Hace que el conjunto de búsquedas sea secuencial en lugar de aleatorio (habitualmente más rápido). Se activa automáticamente al usar -s all.
--	--max-leaf	Número máximo de puntos por hoja del Octree (por defecto = 128). No aplica al Octree de PCL.
--	--pcl-oct-resolution	Tamaño mínimo de octante para la subdivisión en el Octree de PCL.

B.4. Mensajes de error y problemas comunes

A continuación se recogen algunas situaciones de error habituales y cómo resolverlas:

- **El programa no encuentra el fichero de entrada:** comprobar que la ruta indicada en -i/--input es correcta y que el fichero existe, tiene formato .LAS y tiene permisos de lectura.
-